
Proxy Compression for Language Modeling

Lin Zheng^{*1} Xinyu Li^{*1} Qian Liu² Xiachong Feng¹ Lingpeng Kong¹

Abstract

Modern language models are trained almost exclusively on token sequences produced by a fixed *tokenizer*, an external lossless compressor often over UTF-8 byte sequences, thereby coupling the model to that compressor. This work introduces proxy compression, an alternative training scheme that preserves the efficiency benefits of compressed inputs while providing an end-to-end, raw-byte interface at inference time. During training, a single language model is jointly trained on raw byte sequences and compressed views generated by external compressors; through the process, the model learns to internally align compressed sequences and raw bytes. This alignment enables strong transfer between the two formats, even when training predominantly on compressed inputs that are discarded at inference. Extensive experiments on code language modeling demonstrate that proxy compression substantially improves training efficiency and significantly outperforms pure byte-level baselines given fixed compute budgets. As model scale increases, these gains become more pronounced, and proxy-trained models eventually match or surpass tokenizer approaches, all while operating solely on raw bytes and retaining the inherent robustness of byte-level modeling. Our code is available at <https://github.com/LZhengisme/proxy-compression>.

1. Introduction

Modern language models are almost always trained on compressed views of data rather than on the raw format (Bengio et al., 2003). In practice, the compressor, most commonly a tokenizer (Sennrich et al., 2016; Kudo & Richardson, 2018) or arithmetic coding with another language model (Lester

et al., 2024), maps the raw input into a shorter sequence of discrete tokens that the model actually processes. This compression is computationally essential, as it reduces the sequence length and thus enables efficient training.

However, this design couples the entire modeling stack to a fixed external compressor. Every input and output must pass through it, and at the level of raw data,¹ the model is no longer strictly end-to-end, as it learns only to manipulate the compressor’s outputs. In the case of hand-crafted tokenizers, this coupling introduces well-documented artifacts (Phan et al., 2024), including prompt boundary problems (Microsoft, 2023; Lundberg & Ribeiro, 2023; Athiwaratkun et al., 2024; Hayase et al., 2025), retokenization drift (Team, 2025), under-trained or glitch tokens (Rumbelow & Watkins, 2023; Land & Bartolo, 2024; Wang et al., 2024a; Yang, 2024; Yang et al., 2024), data mixture leakage (Hayase et al., 2024), biases against low-resource languages (Petrov et al., 2023; Ahia et al., 2023; Limisiewicz et al., 2024), and increased sensitivity to adversarial inputs (Sun et al., 2020; Pagnoni et al., 2024).

In this work, we ask whether it is possible to keep the efficiency benefits of training on compressed data without hard-wiring the compressor into the model’s interface. We address this with *proxy compression* (Figure 1): treating external compressors as a *training-time proxy* rather than a permanent part of the pipeline. Given a training corpus, we apply an external compressor to produce compressed views of each sequence and mix these compressed sequences with raw UTF-8 counterparts during training. A single language model is trained to perform next-symbol prediction over both representations jointly. At inference time, we can discard the compressor entirely and run the model on raw bytes alone; the compressor has served only to provide shorter proxy sequences during training for efficiency. Our central finding enabling proxy compression is strong *cross-representation transfer*: although the majority of training data is specified in compressed form for efficiency (e.g., 90%), the model performs surprisingly well on raw-byte inference. Moreover, transfer strength of proxy compression grows with model scale: small models demonstrate weak or

^{*}Equal contribution ¹University of Hong Kong ²TikTok. Correspondence to: Lin Zheng <lzheng2@cs.hku.hk>.

¹We use *raw* to refer to the UTF-8 byte encoding of text, contrasting it with compressed sequences often derived from that byte stream, such as tokenized text.

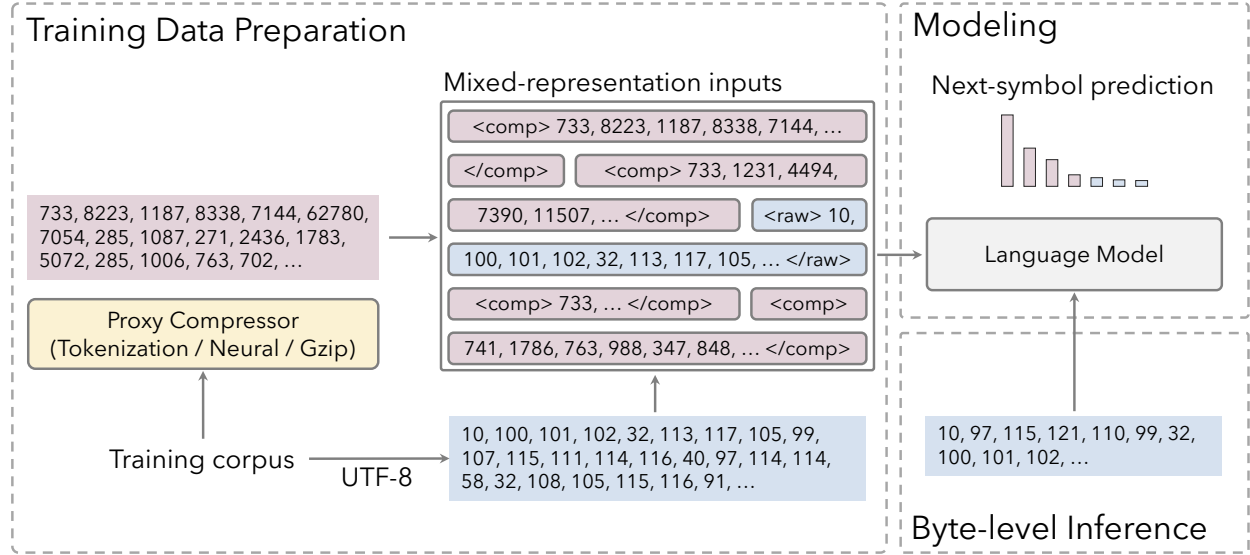


Figure 1. Overview of proxy compression for language modeling. During training, we prepare mixed-representation inputs by combining compressed sequences with raw UTF-8 bytes, which are packed together to train a single language model with next-symbol prediction over both representations. Different representations are associated with special sentinels, such as `<comp>`, `</comp>` for compressed data and `<raw>` and `</raw>` for raw data. At inference time, the proxy compressor is discarded entirely, and the model operates solely on raw bytes. By training primarily on compressed data (e.g., 90% of training data in this work), this approach captures the training efficiency benefits of compressed data without hard-wiring the compressor into the model’s interface.

negative transfer, while larger models with proxy compression can match or surpass ordinary language models with hard-wired compressors (Figure 2).

Conceptually, proxy compression makes the learning objective more demanding in a useful way. The model must predict in two different code spaces and implicitly align them, learning an internal mapping between proxy codes and bytes. This encourages the model to treat compressed sequences as informative hints rather than a complete substitute for the raw byte stream. From a coding perspective, a language model defines a compressor via its predictive distribution (Deletang et al., 2024); proxy compression can thus be viewed as partially delegating compression to an external proxy while training the model to remain an effective compressor over raw bytes. This navigates a fundamental trade-off: training on purely compressed data is efficient but constrains the model to patterns the compressor exposes, while training with pure raw bytes captures all structure but at high computational cost. Proxy compression gains efficiency from compressed inputs while staying grounded in the underlying byte-level distribution.

We instantiate proxy compression with several families of compressors (§2): tokenizer-based compression (§2.2), which uses conventional tokenizers as proxy compressors; neural proxy compressors (§2.3), which combine a separately trained byte-level model with arithmetic coding (Lester et al., 2024); and generic, structure-agnostic compression via gzip (§2.4). Our experiments on code language

modeling (§3) demonstrate that proxy compression yields strong transfer from compressed training to raw-byte inference. On downstream benchmarks (§3.2), models that see only about 10% of their training samples in raw UTF-8 nonetheless outperform pure raw-byte baselines under a fixed compute budget, as they can consume substantially more compressed data while benefiting from strong transfer; at scale, they match or surpass strong tokenizer-based baselines. In-context transfer probes (§3.3) show that proxy-trained models can near-perfectly recover raw inputs from their compressed forms in context. We then investigate what makes a good proxy compressor in §3.4, where gzip proxies fail to transfer, possibly due to their unstable outputs, whereas both tokenizer-based and neural proxies support strong transfer. Robustness evaluations (§3.5) show that proxy-trained models retain most of the inherent robustness of raw byte-level modeling.

In summary, this work makes the following contributions:

- We introduce *proxy compression*, a mixed-representation training scheme that enables efficient training over compressed data while keeping a simple, raw-byte inference interface, without modifying model architectures.
- We demonstrate *strong cross-representation transfer* with proxy compression: models trained predominantly on proxy-compressed inputs substantially outperform byte-level baselines and rival tokenizer-based approaches at scale, with the gap narrowing as the model scale increases.

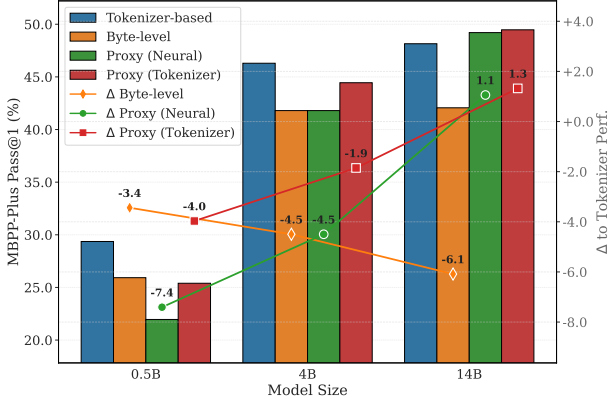


Figure 2. Model performance (Pass@1) on MBPP-Plus across model scales. Bars show absolute performance (left axis); lines show the performance gap (Δ) relative to the tokenizer baseline (right axis). While byte-level models exhibit a persistent or widening gap, proxy-based models progressively close the gap as the model scale increases.

- We systematically study several proxy compressors, including generic gzip, tokenizer-based compression, and arithmetic-coded neural proxies, showing that more structured compressors (tokenizer-based and neural) are highly effective proxies for training language models, while gzip-based compression fails to transfer effectively.

2. Proxy Compression

We introduce *proxy compression*, a training scheme that retains the efficiency of compressed inputs while exposing an end-to-end byte-level interface at inference. §2.1 describes the core framework, §§ 2.2 to 2.4 present concrete instantiations of the proxy compressor, and §2.5 provides implementation details.

2.1. Overview

Let x_{raw} denote a raw input sample in UTF-8 bytes, and let $x_{\text{comp}} := f(x_{\text{raw}})$ be a representation compressed by a proxy compressor f . We assume x_{comp} is a sequence of discrete symbols from a finite vocabulary, so that both formats can be handled seamlessly by a standard language model.

Mixed-Representation Training. Our training pipeline operates at the sample level. For each input x_{raw} , we draw a Bernoulli variable such that with probability r , the sample is presented in compressed form x_{comp} , otherwise it remains as x_{raw} . The resulting sequences are packed into fixed-length contexts, which may contain both formats within the same context, and a single standard autoregressive model is trained with the usual next-symbol prediction objective on both compressed and raw sequences. Crucially, this scheme requires *no architectural changes*; all modifications

are confined to the data preprocessing pipeline.

Byte-level Inference. At inference time, the model operates *exclusively* on raw UTF-8 bytes; the proxy compressor is used only during training and can be discarded afterwards. This decoupling leverages compression for training efficiency while retaining a universal byte-level interface.

Format Sentinel Tokens. To make different representations explicitly distinguishable, we wrap each sequence with special sentinel tokens that indicate its format. Raw and compressed sequences are encoded as $[\langle \text{raw} \rangle \circ x_{\text{raw}} \circ \langle / \text{raw} \rangle]$ and $[\langle \text{comp} \rangle \circ x_{\text{comp}} \circ \langle / \text{comp} \rangle]$, respectively, where $\circ$ denotes concatenation. These markers allow the model to condition its predictions on the representation type.²

In-context Translation Pairing. The sampling scheme above treats raw and compressed samples independently. To further encourage cross-representation alignment, we introduce *in-context translation pairing*, which optionally presents both views of the same sample concatenated within a single context (the ordering between x_{raw} and x_{comp} is randomized with equal probability),

$$[\langle \text{raw} \rangle \circ x_{\text{raw}} \circ \langle / \text{raw} \rangle \circ \langle \text{comp} \rangle \circ x_{\text{comp}} \circ \langle / \text{comp} \rangle].$$

These paired sequences encourage the model to predict one representation conditioned on the other, effectively learning an in-context “translation” between formats. By default, pairing is enabled only during an initial warm-up phase (§2.5); further analysis is provided in Appendix D.8.

Note that mixed-representation training usually yields a lower effective compression rate than the proxy compressor f alone; see Appendix A for more detailed discussion. We next describe concrete instantiations of f that produce discrete symbol sequences amenable to offline preprocessing.

2.2. Tokenizer-based Compressors

A natural instantiation of the proxy compressor f is standard tokenization. Given a trained tokenizer (Sennrich et al., 2016; Kudo & Richardson, 2018; Liu et al., 2025), it segments x_{raw} into token indices from a fixed-size vocabulary to obtain x_{comp} . This satisfies our design criteria: the output is a discrete sequence, and tokenization can be performed entirely offline. Conceptually, the use of tokenization for proxy compression differs from conventional tokenizer-based language model training. Whereas a standard tokenizer-based model operates exclusively in token space and almost never sees raw bytes, our approach instead treats tokens as a *training-time proxy* for efficiency while still retaining a raw-byte interface at inference. We

²Sentinel tokens also improve performance at smaller model scales with diminishing gains at larger scales; see Appendix D.10.

also explored alternative token encodings (e.g., mapping token IDs to fixed-length byte sequences); these variants did not improve over direct token-index representations and are discussed in Appendix B.1.

2.3. Neural Compressors

Beyond tokenization, we study neural proxy compressors based on arithmetic coding (Lester et al., 2024). We train a small byte-level language model ($\sim 40\text{M}$ parameters) on raw UTF-8 data to obtain conditional probability estimates for each byte position of the input x_{raw} . We then apply arithmetic coding with equal-information windows (Lester et al., 2024) to x_{raw} using these probabilities to produce a compressed bitstream, which is then packed into discrete symbols to form x_{comp} .

A naive implementation of this pipeline is prohibitively slow due to the sequential nature of arithmetic coding. To enable neural compressors at scale, we introduce an *entropy-based segmentation* strategy: we use the compressor model to estimate per-byte entropy, identify segment boundaries at high-entropy positions, and compress segments independently in parallel. This is essential for making neural proxy compression practical on large-scale corpora, not only improving compression throughput but also yielding better empirical performance; full details are provided in Appendix B.2.

Neurally Compressed Representations are Fuzzy. Neural compressors behave quite differently from tokenizer-based compressors. Although each raw input x_{raw} maps deterministically to a unique compressed sequence x_{comp} , the reverse mapping is generally *not* unique: given only x_{comp} and no record of the compressing model probabilities used during encoding, there might be multiple distinct raw byte sequences corresponding to the same compressed stream. We refer to the compressed inputs as “fuzzy” in the sense that neural compression induces a many-to-one mapping from raw bytes to compressed symbols.

Importantly, this ambiguity is highly structured rather than arbitrary. As we analyze in §3.4, collisions typically group together raw byte sequences that differ only in superficial, low-entropy details (such as whitespace, newlines, and indentation), while preserving their semantic content. Unlike tokenization, we cannot losslessly decode x_{comp} back to x_{raw} . However, for proxy compression this controlled ambiguity turns out to be beneficial: it abstracts away formatting noise and implicitly shares representations across semantically equivalent patterns.

2.4. Gzip Compressors

We also instantiate f using general-purpose compressors. We apply gzip (via Python’s standard library with `gzip.compress(..., mtime=0)` to remove variation

due to headers and trailers) directly to x_{raw} and treat the output byte stream as x_{comp} . On our corpus, gzip achieves a compression rate of roughly $2.5\times$ over x_{raw} . Unlike tokenizer-based and neural proxies, gzip produces streams whose local patterns depend strongly on document-specific compression state; we later show that this results in substantially less stable representations (§3.4).

2.5. Implementation Details

Mixing Training Schedule. We set the mixing rate $r = 0.9$, so that 90% of training samples are presented in compressed form and only 10% as raw bytes. In this case, tokenizer-based and neural proxy compressors achieve average compression rates of roughly $2.9\times$ and $2.6\times$, respectively. During an initial warm-up phase (first 10k steps), we enable in-context translation pairing (§2.1) and linearly increase r from 0.4 to 0.9; after warm-up, pairing is disabled and r remains fixed at 0.9. We also explored starting from different rates such as $r = 0.1$ and observed no significant difference in final performance. All proxy-trained models are evaluated exclusively on raw byte inputs at inference time unless otherwise noted (§3.6).

Vocabulary. Since input sequences now consist of both compressed and raw sequences, we use a shared vocabulary to accommodate both raw bytes and compressed symbols, partitioned as follows: the first 64 indices in the vocabulary are reserved for special sentinel tokens, followed by 256 entries for raw UTF-8 byte values, with remaining indices assigned to compressed symbols. For tokenizer-based proxy compressors, we use the OpenCoder tokenizer vocabulary (Huang et al., 2024a) with 96,640 symbols. For neural compression, we pack every 16 bits into a compressed symbol (a 65,536-way alphabet). The gzip-based proxy compressor works at the byte level over 256 compressed symbols.

3. Experiments

In this section, we empirically evaluate proxy compressed inputs. We first describe the experimental setup in §3.1, study transfer from proxy compressors at scale in §§3.2 and 3.3, compare different proxy compressors (§3.4), and finally evaluate the robustness of our approach in §3.5.

3.1. Setup

We center our empirical study on code for compute-matched comparisons across input representations; complementary natural-language experiments at 1.5B parameters (Appendix D.12) verify that the transfer mechanism is not domain-specific. All training experiments use the RefineCode corpus (Huang et al., 2024a), where we primarily use its Python subset, totaling roughly 270 GB of Python source code, and its full GitHub split (used in §3.2) with

Table 1. Downstream pass@1 performance on HumanEval-Plus and MBPP-Plus across different model sizes and input representations. CR denotes compression rate (avg. bytes per symbol). All models are trained with a fixed budget of 100B symbols (tokens or bytes); larger models (7B, 14B) may be undertrained relative to compute-optimal scaling (Hoffmann et al., 2022).

Task	Model	CR	Model Size					
			0.5B	1.5B	4B	7B	14B	
HumanEval-Plus	Tokenizer-based	3.7	17.7	18.3	28.0	28.7	29.3	
	Byte-level	1.0	15.9	18.3	22.0	23.8	24.4	
	Proxy (Neural)	2.6	13.4	18.3	22.6	26.8	29.9	
	Proxy (Tokenizer)	2.9	12.2	20.7	24.4	26.2	30.5	
MBPP-Plus	Tokenizer-based	3.7	29.4	41.0	46.3	45.2	48.1	
	Byte-level	1.0	25.9	33.6	41.8	41.3	42.1	
	Proxy (Neural)	2.6	22.0	29.6	41.8	41.8	49.2	
	Proxy (Tokenizer)	2.9	25.4	38.4	44.4	45.5	49.5	

approximately 3.3 TB of code across multiple programming languages. We train a family of language models following the EvaByte architecture (Zheng et al., 2025b) at 0.5B, 1.5B, 4B, 7B, and 14B parameters for all input representations. We choose EvaByte because it provides both strong modeling quality and efficient byte-level decoding: our architecture ablation shows that EvaByte achieves lower validation BPB while matching OpenCoder (Huang et al., 2024a), a Llama-based transformer architecture (Dubey et al., 2024), on downstream code generation (Figure 10), and its efficient attention and multi-byte prediction mitigate the longer sequences induced by byte-level inference. Training runs for 50K steps with a fixed batch size of 2M sequence symbols (e.g., UTF-8 bytes, tokens, etc.); this yields a comparable training FLOPs budget across representations. As a result, models operating on different representations can consume different amounts of raw data per step. We compare against two baselines: (i) a *byte-level* model trained entirely on raw UTF-8 bytes, and (ii) a *tokenizer-based* model trained on BPE tokens using the OpenCoder tokenizer (Huang et al., 2024a). For evaluation, we focus on code generation tasks including HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and their EvalPlus variants (Liu et al., 2023). Full experimental details are provided in Appendix C.

3.2. Main Results: Scalable Downstream Transfer

In this section, we address the central question: *can we train primarily on proxy-compressed inputs yet deploy the model purely on raw bytes, without ever exposing compressed formats at inference time?* Concretely, we evaluate whether mixed-representation training with proxy compressors yields effective transfer to raw byte streams on downstream code generation tasks.

Transfer Scales with Model Size. As shown in Table 1, proxy-compressed inputs yield strong and scalable transfer on downstream tasks. Despite observing raw bytes on only 10% of training samples, proxy-trained byte models

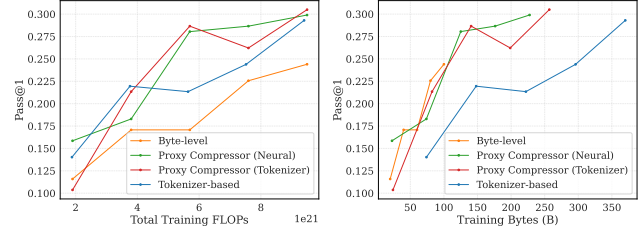


Figure 3. Pass@1 performance on HumanEval-Plus for 14B models under different input representations, compared as a function of training FLOPs (left) and amount of training data (right).

closely track or outperform the fully byte-level baseline at model sizes above 1.5B parameters. This indicates that compressed views effectively compensate for the significantly reduced raw-byte exposure by consuming more data and that cross-representation transfer is strong enough to recover most of the performance of fully byte-level training. Moreover, the advantage of proxy training becomes more pronounced with scale. At larger model sizes, proxy-trained models not only significantly outperform byte-level baselines, but also start to achieve or surpass the performance of tokenizer-based models. This can be attributed to the greater capacity of larger models to encode the alignment between compressed and raw views in their weights. Both neural and tokenizer-based proxies perform competitively, with neural compression achieving slightly lower compression ($2.6\times$) than the tokenizer-based compressor ($2.9\times$), yet still matching its performance at scale.

Data versus Compute Efficiency. Byte-level and tokenizer-based models exhibit a well-known trade-off (Xue et al., 2022; Zheng et al., 2025b): tokenizer-based models are more compute-efficient (better performance under a fixed FLOPs budget, as they consume more data per unit of compute), while byte models are more data-efficient (better performance under the same amount of training data, as they allocate more optimization steps and thus more compute per training example).³

Under proxy compression, this trade-off shifts systematically with scale. At 0.5B-7B (Figures 19a to 19d), byte-level training remains more data-efficient, while proxy models generally lie between the two baselines and progressively narrow the gap. At 14B (Figure 3), proxy compression is able to capture the best of both regimes, performing comparably to tokenizer baselines under matched FLOPs and retaining byte-level data efficiency while substantially outperforming tokenizer baselines under matched data.

Longer Training Horizons. To test whether these trends hold at larger scale, we extend training to 320B symbols

³This trade-off does not always hold when varying the scale and training horizon; please see Appendix D.7 for further analysis.

Table 2. Downstream pass@1 on HumanEval-Plus and MBPP-Plus after training 320B sequence symbols on the full RefineCode GitHub data.

# Parameters	Model	HumanEval-Plus	MBPP-Plus
1.5B	Tokenizer-based	17.1	28.0
	Byte-level	9.1	23.3
	Proxy (Neural)	14.0	24.1
	Proxy (Tokenizer)	12.8	25.1
7B	Tokenizer-based	21.3	36.0
	Byte-level	14.6	32.5
	Proxy (Neural)	21.3	36.0
	Proxy (Tokenizer)	22.0	34.9

(tokens or bytes) on the full RefineCode GitHub split, following the OpenCoder setup (Huang et al., 2024a). Table 2 confirms consistent patterns: at 1.5B parameters, proxy models outperform byte baselines but still trail tokenizer baselines; at 7B, they close the gap and often match or exceed tokenizer-based models. This shows that proxy compression remains effective under longer training horizons.

Overall, these results show that proxy-compressed training enables efficient learning from compressed data, while achieving *scalable downstream transfer* to raw-byte inference, with benefits that amplify at scale.

3.3. In-context Cross-Representation Transfer

The results in §3.2 demonstrate *in-weight transfer*: knowledge learned from compressed inputs transfers to raw-byte inference through the model parameters. Here, we probe a more explicit form of transfer called *in-context translation*, where both compressed and raw views of the *same* input appear in a single context, and the model must translate between representations on the fly. For each problem on HumanEval-Plus (Chen et al., 2021; Liu et al., 2023), we take the raw prompt p_{raw} and oracle solution s_{raw} , compress them into p_{comp} and s_{comp} , and construct a mixed-representation prompt

$$[\langle \text{comp} \rangle \circ p_{\text{comp}} \circ s_{\text{comp}} \circ \langle / \text{comp} \rangle \circ \langle \text{raw} \rangle \circ p_{\text{raw}}].$$

The model must decode the corresponding raw solution s_{raw} in raw bytes. We report *oracle-translation pass@1*: whether the model recovers the correct solution given its compressed form in context. To study how in-context transfer depends on explicit pairing as described in §2.1, we compare three training schedules (all with mixing ratio $r=0.9$): *No pairs* (independent sampling without any paired data), *Warmup-only* (enabling translation pairs for first 10k training steps only), and *Always-on* (pairs throughout training). More details are in Appendix D.2.

Table 3 reports both ordinary (i.e., no-oracle) and oracle-translation pass@1. Mixed-representation training induces substantial in-context transfer even without explicit pairs: under *No pairs*, oracle-translation pass@1 reaches $\sim 46\%$

Table 3. Ordinary pass@1 (%) at 50k training steps and oracle-translation pass@1 (%) at 10k, 20k, 30k, 40k, and 50k training steps on HumanEval-Plus for different compressor types and pairing schedules (1.5B model; higher is better). We highlight cells with pass rates higher than 90%.

Compressor Type	Model Variant	Pass@1 50k steps	Oracle Translation Pass@1				
			10k	20k	30k	40k	50k
Tokenizer	No pairs	17.0	7.30	29.3	40.2	44.5	45.7
	Warmup-only	20.7	90.9	31.1	42.1	53.7	45.7
	Always-on	17.0	93.3	95.7	96.3	96.3	96.3
Neural	No pairs	14.6	0.6	3.0	7.9	20.7	32.9
	Warmup-only	18.9	90.9	39.0	14.6	22.0	38.4
	Always-on	14.6	94.5	92.1	93.9	95.1	95.1

(tokenizer) and $\sim 33\%$ (neural), well above no-oracle baselines. With *Always-on* pairing, both compressors rapidly achieve near-perfect translation ($>95\%$ pass@1), demonstrating that even ambiguous neural compression can be reliably disambiguated in context. Under *Warmup-only*, translation accuracy decays toward the *No pairs* baseline once pairing is removed, yet still outperforming no-oracle baselines, indicating retained transfer capability.

Interestingly, high translation accuracy is not necessary for strong downstream performance (e.g., §3.2, which uses *Warmup-only* by default). Comparing *Warmup-only* and *Always-on*, we observe a significant drop in oracle-translation accuracy, yet ordinary pass@1 is slightly better for *Warmup-only*. Two confounded factors plausibly contribute: *Always-on* duplicates each sample as a pair and thus sees fewer unique documents under matched FLOPs (Appendix A), and persistent co-presence of both views may encourage reliance on in-context translation, which is absent when only one view is available at inference. The performance of *Warmup-only* is consistent with transfer operating beyond literal sequence-to-sequence translation: the model can still extract and leverage shared structure across representations *in-weight* even without maintaining perfect in-context translation. Extended analyses are provided in Appendix D.2.

3.4. What Makes a Good Proxy Compressor?

The results above show that tokenizer-based and neural proxies enable strong transfer. We now broaden the comparison to include gzip proxies and ask: what distinguishes effective proxy compressors from ineffective ones? We hypothesize that useful proxy representations should be structurally regular: small changes to the raw input should not cause disproportionate changes to the compressed sequence, and recurring syntactic or semantic patterns should induce recurring compressed patterns. We test this hypothesis by measuring compressor stability: we apply random 10% character deletion to 80K samples drawn from training data and compute the normalized Levenshtein distance between compressed outputs before and after the perturbation. As

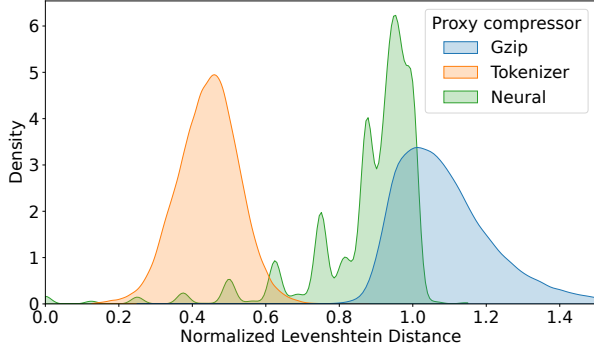


Figure 4. Compressor stability analysis under input perturbation: we apply random 10% character deletion to 80K samples and measure normalized Levenshtein distance between compressed outputs before and after perturbation.

shown in Figure 4, tokenization is highly stable, gzip is significantly more unstable (small edits cause large output changes), and neural compression lies in between.

Gzip Proxies Fail to Transfer. Figure 5 shows downstream performance for 1.5B models trained with varying gzip-versus-raw ratios. Unlike tokenizer and neural proxies, increasing the proportion of gzip compressed data *degrades* performance: models trained on pure raw bytes (0% gzip) always outperform gzip-mixed variants. This indicates weak or even negative transfer from gzip-compressed sequences to raw bytes. We attribute this failure to the highly unstable and unstructured outputs from gzip (Figure 4), where small input perturbations cause drastic changes in the compressed sequence, making it difficult for the model to learn consistent patterns. In addition, gzip exploits low-level byte redundancies *per sample* without respecting semantics, producing outputs that resemble noise to a language model. Empirically, we observe that models trained on gzip-compressed data fail to complete partial gzip sequences coherently, suggesting they never learn the underlying compression scheme.

Neural Compression: Structured Fuzziness. As discussed in §2.3, neural compression is non-invertible: a single compressed sequence may correspond to multiple raw byte chunks (*collisions*). Despite the ambiguity and unstable mapping (Figure 4), neural proxies still achieve strong transfer. We analyze this in Figure 6, showing that most compressed segments exhibit collisions. To quantify similarity among colliding chunks, we compute the *longest common prefix (LCP) ratio*: the length of the shared prefix divided by average chunk length. Crucially, these collisions are highly structured: over 90% of collisions have LCP ratios above 0.8, meaning most colliding chunks are nearly identical except for short suffixes (Figure 12), likely because the suffix entropies are too low for the arithmetic coder to

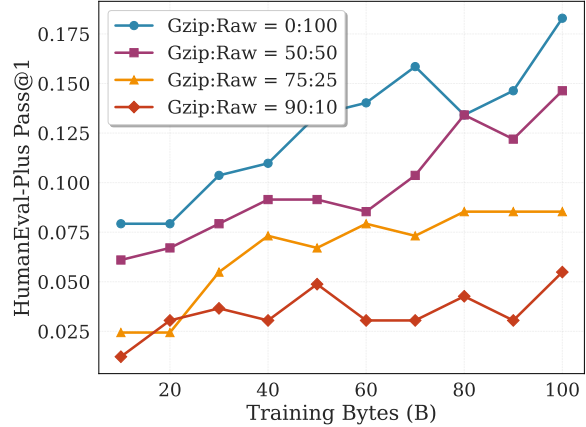


Figure 5. HumanEval-Plus pass@1 of 1.5B gzip-proxy models with different mixing ratios r as a function of training data.

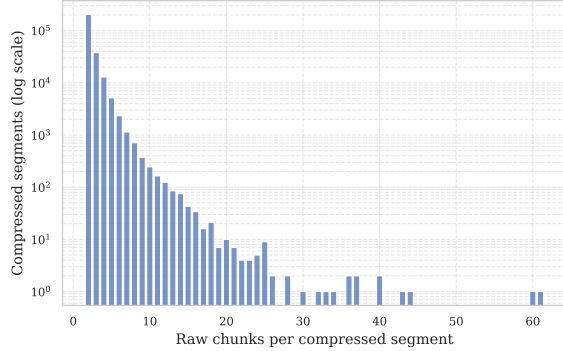


Figure 6. Collision statistics for the neural compressor. The x -axis is the number of distinct byte chunks that collide on the same compressed segment, and the y -axis is the number of such compressed segments in log scale.

allocate additional bits. This *structured fuzziness* abstracts away formatting noise while preserving semantics, potentially explaining why neural proxies match tokenizer-based proxies despite their ambiguity. See Appendix D.3 for a detailed analysis.

3.5. Robustness Evaluation

Byte-level models have been shown to be more robust to input perturbations than tokenizer-based models (Pagnoni et al., 2024; Hwang et al., 2025). In this section, we investigate whether proxy-trained models inherit this advantage. We evaluate robustness on the HumanEval split of ReCode (Wang et al., 2023), which applies semantics-preserving perturbations to coding problems in four families: function name rewrites (*Function*), formatting changes (*Format*), syntactic rewrites (*Syntax*), and docstring paraphrases (*Docstrings*). We report standard pass@1 on unperturbed inputs and *Robust Pass@1* (RP), the worst-case pass@1 across 5 perturbed variants per problem. Full metric definitions and per-family breakdowns are in Appendix D.4.

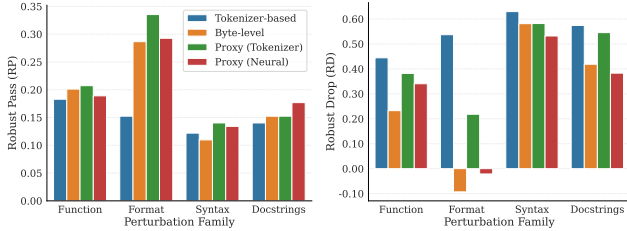


Figure 7. Robust pass (left, higher is better) and robust drop (right, lower is better) performance on ReCode per perturbation family with different models.

Table 4. Robustness evaluation on the HumanEval split of ReCode with 7B models. We report standard pass@1 and robust pass@1 (RP), the macro average across perturbation families.

Model	Pass@1	Robust Pass@1 ($\overline{RP}$) $\uparrow$
Tokenizer-based	32.9	14.9
Byte-level	26.2	18.7
Proxy (Tokenizer)	32.9	19.1
Proxy (Neural)	30.5	19.8

Table 4 reports standard pass@1 and macro-averaged robust pass@1 ($\overline{RP}$). Although the byte-level model achieves lower no-perturbation pass@1, it exhibits much better robustness than tokenizer baselines ($\overline{RP}$: 18.7 vs. 14.9). Proxy-trained models inherit and amplify these robustness advantages: neural proxies attain the highest $\overline{RP}$, while tokenizer proxies substantially improve robustness over the tokenizer baseline. One plausible explanation is that the fuzzy representations induced by neural compression (§3.4) help smooth over superficial noise, encouraging the model to focus on invariant structure during training. Figure 7 reveals that *Format* perturbations drive the largest gap: byte-level and proxy models remain near-nominal under whitespace and indentation changes, whereas the tokenizer baseline degrades sharply. On *Syntax*, all models suffer a large robust drop (RD), though proxy models still improve RP. Overall, despite being trained predominantly on compressed inputs, our proxy-compressed models can mostly retain and in some cases improve upon the robustness of byte-level models; we further verify this with a targeted prompt-boundary study in Appendix D.11.

3.6. Analyses

Inference-Time Interface. For tokenizer-based proxy compression, we can perform inference on either raw bytes or compressed tokens. As shown in Figure 8, byte-level inference in many cases matches or outperforms token-level inference, though results vary across benchmarks and compute budgets. This is notable because only 10% of training samples are presented in raw-byte form. We attribute this to two factors: (i) cross-representation transfer is sufficiently strong that the model performs well on raw bytes even with

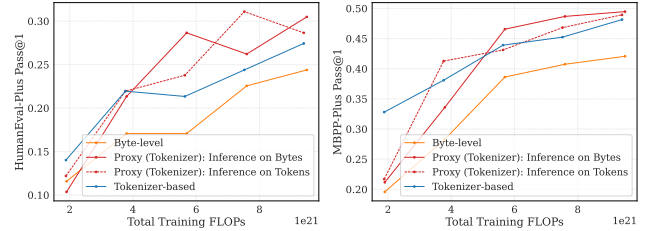


Figure 8. Inference-time interface comparison on HumanEval-Plus (left) and MBPP-Plus (right) for 14B tokenizer-proxy models.

limited exposure, and (ii) longer byte sequences afford more test-time compute per problem instance. For neural compression, we report only raw-byte inference: its compressed symbols are training-time proxies and are not directly decodable into text, so they do not define a usable inference interface (§2.3).

Mixing Ratio Ablation. Throughout our experiments, we fix $r = 0.9$ as the proportion of compressed samples in the training corpus. To study the effect of raw-compressed data composition in proxy compression training, we train 1.5B parameter models on mixtures containing varying proportions of raw and compressed data, maintaining a constant budget of 50000 training steps. Figure 9 reveals that byte-level inference performance does not degrade monotonically as the proportion of compressed samples increases. This pattern is explained by the document count curve (dash-dot line), which shows that reducing the raw-byte proportion in the mixture increases the number of unique training samples seen within a fixed compute budget. At $r=0.9$, models observe approximately $3\times$ more samples than at $r=0.0$ (100% raw bytes), and strong compressed-to-raw transfer allows them to benefit from this additional data. The mixing ratio ablation also reveals *an asymmetry in transfer direction*. For byte-level inference (solid line in Figure 9), performance remains strong even with only 10% raw data, indicating robust transfer from compressed to raw representations. In contrast, for token-level inference (dashed line), performance degrades nearly monotonically as the raw-byte proportion increases, suggesting weak transfer from raw back to compressed representations.

4. Related Work

Input Representations for Language Models. Most modern language models rely on external tokenization that segments textual data into sequences of discrete tokens, such as Byte-Pair Encoding (Gage, 1994; Sennrich et al., 2016), WordPiece (Schuster & Nakajima, 2012; Wu et al., 2016; Devlin et al., 2019), UnigramLM (Kudo, 2018), and SentencePiece implementations (Kudo & Richardson, 2018). Tokenization compresses raw text into shorter token sequences, and recent work has pushed this design along multiple axes,

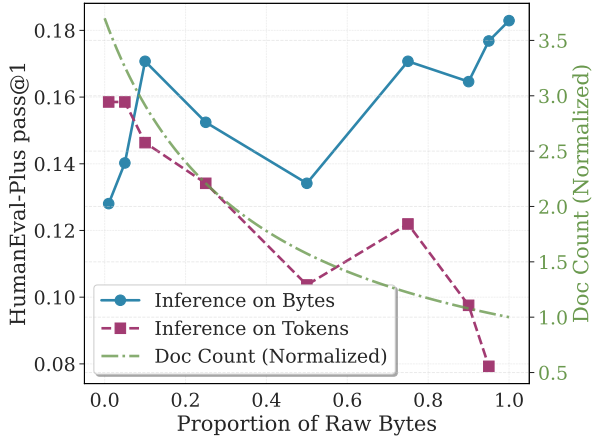


Figure 9. Performance on HumanEval-Plus Pass@1 as a function of raw byte proportion in the training mixture for 1.5B models with tokenizer-based proxy compression. Solid and dashed lines show models performing byte-level and token-level inference, respectively. The dash-dot line (right y-axis) indicates the normalized document count seen during training within a fixed training FLOPs budget.

including analyzing the overall tokenization pipeline design (Bostrom & Durrett, 2020; Zouhar et al., 2023; Schmidt et al., 2024; Dagan et al., 2024), studying non-canonical tokenization behavior (Cao & Rimell, 2021; Geh et al., 2024; 2025; Zheng et al., 2025a), exploring vocabulary scaling (Tao et al., 2024; Yu et al., 2025; Huang et al., 2025), and improving compression (Fried et al., 2023; Gee et al., 2023; Schmidt et al., 2025; Liu et al., 2025).

Besides tokenization, recent work also explores distinct approaches to constructing compressed representations, such as gzip-based compressors (Jiang et al., 2023; Lester et al., 2024), arithmetic coding with equal-information windows (Lester et al., 2024), concept-level semantic units (Barrault et al., 2024; Qu et al., 2025), morphology-driven byte representations (Limisiewicz et al., 2024) that improve efficiency and fairness in multilingual settings, and pixel-rendered text (Salesky et al., 2021; Rust et al., 2023; Salesky et al., 2023; Li et al., 2023; Lee et al., 2023; Lotz et al., 2023; Tai et al., 2024; Gao et al., 2024; Wei et al., 2025; Cheng et al., 2025).

Byte-level Models. Alternatively, one can directly use byte-level sequences as the input representation (Sutskever et al., 2011; Graves, 2013; Radford et al., 2017; Chung et al., 2017; Hwang & Sung, 2017; Al-Rfou et al., 2019; Choe et al., 2019; Xue et al., 2022; Wang et al., 2024b; Neitemeier et al., 2025; Zheng et al., 2025b; Minixhofer et al., 2025). To mitigate the computational cost of longer sequences, recent work aims to pool input sequences into shorter representations inside the model architecture, including downsampling the input in a context-independent manner (Jaegle et al., 2021b;a; Hawthorne et al., 2022; Clark et al., 2022; Nawrot et al., 2022; Yu et al., 2023; Pagnoni

et al., 2024; Slagle, 2024; Videau et al., 2025). Recent work also explores adaptive compression with dynamically adjusted granularity based on the input content (Tay et al., 2022; Godey et al., 2022; Nawrot et al., 2023; Kallini et al., 2024; Ahia et al., 2024; Owodunni et al., 2025; Geng et al., 2025; Hwang et al., 2025; Minixhofer et al., 2025). Beyond text-only language modeling, byte-level models are also used for multi-modal learning (Egli et al., 2025; Zheng et al., 2025b). More generally, byte-level models can be used to process various types of digital file contents (Park & Johnson, 2023; Wu et al., 2024; Pérez et al., 2024; Han et al., 2024; Horton et al., 2024; Jolicoeur-Martineau & Gervais, 2025; Li & Chen, 2025; Alcazar et al., 2025).

Our work differs from prior approaches by leveraging compressed representations as a proxy during training while still retaining a byte-level interface at inference, without architectural modifications.

5. Conclusion

We introduced proxy compression, a mixed-representation training scheme that decouples the efficiency benefits of compressed training from the inference-time interface. By jointly training on raw inputs and compressed views produced by external compressors, models learn to align representations and transfer effectively from compressed inputs to raw-byte inference. Extensive experiments on code language modeling demonstrate that proxy-trained models substantially outperform pure byte-level baselines under fixed compute budgets, and at scale match or surpass tokenizer-based models, all while operating solely on raw bytes. Our systematic study of proxy compressors further reveals that tokenizer-based and neural proxies support strong transfer, whereas generic compression (gzip) fails to provide useful training signal, suggesting that structured, semantically meaningful compression is key to effective proxy training.

Limitations. Our evaluation primarily focuses on code language modeling; our natural-language results provide initial evidence beyond code, but larger-scale natural-language, multilingual, and mixed-domain validation remains future work. Because inference is performed on raw bytes, deployment efficiency depends on the underlying byte-level model architecture; EvaByte (Zheng et al., 2025b) mitigates this cost in our experiments, but proxy compression itself does not remove the general challenge of efficient byte-level decoding. The trade-offs among compression rate, transfer strength, and compute efficiency are also not yet fully characterized (e.g., more aggressive compression may amplify efficiency gains but could degrade transfer quality). Finally, incorporating proxy compression directly into model architecture design, rather than treating it purely as a data preprocessing step, may unlock further performance or efficiency improvements.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D., Smith, N., and Tsvetkov, Y. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.614/>.
- Ahia, O., Kumar, S., Gonen, H., Hoffman, V., Limisiewicz, T., Tsvetkov, Y., and Smith, N. A. Magnet: Improving the multilingual fairness of language models with adaptive gradient-based tokenization. *arXiv preprint arXiv:2407.08818*, 2024.
- Al-Rfou, R., Choe, D., Constant, N., Guo, M., and Jones, L. Character-level language modeling with deeper self-attention. In *Proceedings of the AAAI conference on artificial intelligence*, 2019.
- Alcazar, J. C. L., Soldan, M., Saatiatoruji, M., Pardo, A., Itani, H., Perez, J. C., and Ghanem, B. Transformers from compressed representations. *arXiv preprint arXiv:2510.23665*, 2025.
- Athiwaratkun, B., Wang, S., Shang, M., Tian, Y., Wang, Z., Gonugondla, S. K., Gouda, S. K., Kwiatowski, R., Nallapati, R., and Xiang, B. Token alignment via character matching for subword completion. *arXiv preprint arXiv:2403.08688*, 2024.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Barrault, L., Duquenne, P.-A., Elbayad, M., Kozhevnikov, A., Alastruey, B., Andrews, P., Coria, M., Couairon, G., Costa-jussà, M. R., Dale, D., et al. Large concept models: Language modeling in a sentence representation space. *arXiv preprint arXiv:2412.08821*, 2024.
- Bengio, Y., Ducharme, R., Vincent, P., and Jauvin, C. A neural probabilistic language model. *Journal of machine learning research*, 2003.
- Bostrom, K. and Durrett, G. Byte pair encoding is sub-optimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. URL <https://aclanthology.org/2020.findings-emnlp.414/>.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=PEpbUobfJv>.
- Cao, K. and Rimell, L. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021. URL <https://aclanthology.org/2021.emnlp-main.161/>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cheng, J., Liu, Y., Zhang, X., Fei, Y., Hong, W., Lyu, R., Wang, W., Su, Z., Gu, X., Liu, X., et al. Glyph: Scaling context windows via visual-text compression. *arXiv preprint arXiv:2510.17800*, 2025.
- Choe, D., Al-Rfou, R., Guo, M., Lee, H., and Constant, N. Bridging the gap for tokenizer-free language models. *arXiv preprint arXiv:1908.10322*, 2019.
- Chung, J., Ahn, S., and Bengio, Y. Hierarchical multiscale recurrent neural networks. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Sldi0sfgl>.
- Clark, J. H., Garrette, D., Turc, I., and Wieting, J. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 2022. URL <https://aclanthology.org/2022.tacl-1.5>.
- Dagan, G., Synnaeve, G., and Roziere, B. Getting the most out of your tokenizer for pre-training and domain adaptation. *arXiv preprint arXiv:2402.01035*, 2024.
- Deletang, G., Ruoss, A., Duquenne, P.-A., Catt, E., Genewein, T., Mattern, C., Grau-Moya, J., Wenliang, L. K., Aitchison, M., Orseau, L., Hutter, M., and Veness, J. Language modeling is compression. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=jznbginyus>.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for*

- Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423>.
- Diao, S., Yang, Y., Fu, Y., Dong, X., SU, D., Kliegl, M., CHEN, Z., Belcak, P., Suhara, Y., Yin, H., Patwary, M., Lin, Y. C., Kautz, J., and Molchanov, P. Nemotron-CLIMB: Clustering-based iterative data mixture bootstrapping for language model pre-training. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. URL <https://openreview.net/forum?id=aBlqKPkc4a>.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Egli, E., Manica, M., and Born, J. Multiscale byte language models—a hierarchical architecture for causal million-length sequence modeling. *arXiv preprint arXiv:2502.14553*, 2025.
- Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, S., Zettlemoyer, L., and Lewis, M. Incoder: A generative model for code infilling and synthesis. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=hQwb-lbM6EL>.
- Fujii, K., Tajima, Y., Mizuki, S., Shimada, H., Shiotani, T., Saito, K., Ohi, M., Kawamura, M., Nakamura, T., Okamoto, T., et al. Rewriting pre-training data boosts llm performance in math and code. *arXiv preprint arXiv:2505.02881*, 2025.
- Gage, P. A new algorithm for data compression. *C Users Journal*, 1994.
- Gao, T., Wang, Z., Bhaskar, A., and Chen, D. Improving language understanding from screenshots. *arXiv preprint arXiv:2402.14073*, 2024.
- Gee, L., Rigutini, L., Ernandes, M., and Zugarini, A. Multiword tokenization for sequence compression. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2023. URL <https://aclanthology.org/2023.emnlp-industry.58/>.
- Geh, R., Zhang, H., Ahmed, K., Wang, B., and Van Den Broeck, G. Where is the signal in tokenization space? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. URL <https://aclanthology.org/2024.emnlp-main.230/>.
- Geh, R., Shao, Z., and Van Den Broeck, G. Adversarial tokenization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025. URL <https://aclanthology.org/2025.acl-long.1012/>.
- Geng, S., Ranchin, N., Yao, Y., Peyrard, M., Wendler, C., Gastpar, M., and West, R. zip2zip: Inference-time adaptive tokenization via online compression. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=HmepilFm2g>.
- Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- Godey, N., Castagné, R., de la Clergerie, É., and Sagot, B. MANTa: Efficient gradient-based tokenization for end-to-end robust language modeling. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022. URL <https://aclanthology.org/2022.findings-emnlp.207>.
- Graves, A. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- Grivas, A., Loconte, L., van Krieken, E., Nawrot, P., Zhao, Y., Wielewski, E., Minervini, P., Ponti, E., and Vergari, A. Fast and expressive multi-token prediction with probabilistic circuits, 2025.
- Gu, Y., Tafjord, O., Kuehl, B., Haddad, D., Dodge, J., and Hajishirzi, H. OLMES: A standard for language model evaluations. *arXiv preprint arXiv:2406.08446*, 2024.
- Han, X., Ghazvininejad, M., Koh, P. W., and Tsvetkov, Y. Jpeg-lm: Llms as image generators with canonical codec representations. *arXiv preprint arXiv:2408.08459*, 2024.
- Hawthorne, C., Jaegle, A., Cangea, C., Borgeaud, S., Nash, C., Malinowski, M., Dieleman, S., Vinyals, O., Botvinick, M., Simon, I., Sheahan, H., Zeghidour, N., Alayrac, J.-B., Carreira, J., and Engel, J. General-purpose, long-context autoregressive modeling with perceiver AR. In *Proceedings of the 39th International Conference on Machine Learning*, pp. 8535–8558, 2022. URL <https://proceedings.mlr.press/v162/hawthorne22a.html>.
- Hayase, J., Liu, A., Choi, Y., Oh, S., and Smith, N. A. Data mixture inference: What do bpe tokenizers reveal about their training data? *arXiv preprint arXiv:2407.16607*, 2024.

- Hayase, J., Liu, A., Smith, N. A., and Oh, S. Sampling from your language model one byte at a time. *arXiv preprint arXiv:2506.14123*, 2025.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rygGQyrFvH>.
- Horton, M., Mehta, S., Farhadi, A., and Rastegari, M. Bytes are all you need: Transformers operating directly on file bytes. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=RkaqxXAOfN>.
- Huang, H., Zhu, D., Wu, B., Zeng, Y., Wang, Y., Min, Q., and Xun, Z. Over-tokenized transformer: Vocabulary is generally worth scaling. In *Proceedings of the 42nd International Conference on Machine Learning*, pp. 26261–26282, 2025. URL <https://proceedings.mlr.press/v267/huang25bb.html>.
- Huang, S., Cheng, T., Liu, J. K., Hao, J., Song, L., Xu, Y., Yang, J., Liu, J., Zhang, C., Chai, L., et al. Opencoder: The open cookbook for top-tier code large language models. *arXiv preprint arXiv:2411.04905*, 2024a.
- Huang, Y., Zhang, J., Shan, Z., and He, J. Compression represents intelligence linearly. In *First Conference on Language Modeling*, 2024b. URL <https://openreview.net/forum?id=SHMj84U5SH>.
- Hwang, K. and Sung, W. Character-level language modeling with hierarchical recurrent neural networks. In *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 5720–5724. IEEE, 2017.
- Hwang, S., Wang, B., and Gu, A. Dynamic chunking for end-to-end hierarchical sequence modeling. *arXiv preprint arXiv:2507.07955*, 2025.
- Jaegle, A., Borgeaud, S., Alayrac, J.-B., Doersch, C., Ionescu, C., Ding, D., Koppula, S., Zoran, D., Brock, A., Shelhamer, E., et al. Perceiver io: A general architecture for structured inputs & outputs. *arXiv preprint arXiv:2107.14795*, 2021a.
- Jaegle, A., Gimeno, F., Brock, A., Vinyals, O., Zisserman, A., and Carreira, J. Perceiver: General perception with iterative attention. In *Proceedings of the 38th International Conference on Machine Learning*, 2021b. URL <https://proceedings.mlr.press/v139/jaegle21a.html>.
- Jiang, Z., Yang, M., Tsirlin, M., Tang, R., Dai, Y., and Lin, J. “low-resource” text classification: A parameter-free classification method with compressors. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023. URL <https://aclanthology.org/2023.findings-acl.426>.
- Jolicoeur-Martineau, A. and Gervais, E. Bytecraft: Generating video games and animations through bytes. *Preprints*, 2025. doi: 10.20944/preprints202503.1962.v1. URL <https://www.preprints.org/manuscript/202503.1962/v1>.
- Kallini, J., Murty, S., Manning, C. D., Potts, C., and Csordás, R. Mrt5: Dynamic token merging for efficient byte-level language models. *arXiv preprint arXiv:2410.20771*, 2024.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Kudo, T. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018. URL <https://aclanthology.org/P18-1007/>.
- Kudo, T. and Richardson, J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018. URL <https://aclanthology.org/D18-2012/>.
- Land, S. and Bartolo, M. Fishing for magikarp: Automatically detecting under-trained tokens in large language models. *arXiv preprint arXiv:2405.05417*, 2024.
- Lee, K., Joshi, M., Turc, I. R., Hu, H., Liu, F., Eisenschlos, J. M., Khandelwal, U., Shaw, P., Chang, M.-W., and Toutanova, K. Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://proceedings.mlr.press/v202/lee23g.html>.
- Lester, B., Lee, J., Alemi, A., Pennington, J., Roberts, A., Sohl-Dickstein, J., and Constant, N. Training llms over neurally compressed text. *arXiv preprint arXiv:2404.03626*, 2024.
- Li, J., Zhao, W. X., Nie, J.-Y., and Wen, J.-R. Glyphdiffusion: Text generation as image generation. *arXiv preprint arXiv:2304.12519*, 2023.

- Li, Y. and Chen, Z. Bytogen: A tokenizer-free generative model for orderbook events in byte space. *arXiv preprint arXiv:2508.02247*, 2025.
- Limisiewicz, T., Blevins, T., Gonen, H., Ahia, O., and Zettlemoyer, L. Myte: Morphology-driven byte encoding for better and fairer multilingual language modeling. *arXiv preprint arXiv:2403.10691*, 2024.
- Liu, A., Hayase, J., Hofmann, V., Oh, S., Smith, N. A., and Choi, Y. SuperBPE: Space travel for language models. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=lcDRvffeNP>.
- Liu, J., Xia, C. S., Wang, Y., and ZHANG, L. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=1qv610Cu7>.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Lotz, J., Salesky, E., Rust, P., and Elliott, D. Text rendering strategies for pixel language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.628>.
- Lundberg, S. and Ribeiro, M. T. The art of prompt design: Prompt boundaries and token healing. <https://medium.com/data-science/the-art-of-prompt-design-prompt-boundaries-and-token-healing-3b2448b0be38>, 2023. Medium.
- Microsoft. Guidance, 2023. URL <https://github.com/microsoft/guidance>.
- Minixhofer, B., Murray, T., Limisiewicz, T., Korhonen, A., Zettlemoyer, L., Smith, N. A., Ponti, E. M., Soldaini, L., and Hofmann, V. Bolmo: Byteifying the next generation of language models. *arXiv preprint arXiv:2512.15586*, 2025.
- Molybog, I., Albert, P., Chen, M., DeVito, Z., Esiobu, D., Goyal, N., Koura, P. S., Narang, S., Poulton, A., Silva, R., et al. A theory on adam instability in large-scale machine learning. *arXiv preprint arXiv:2304.09871*, 2023.
- Nawrot, P., Tworowski, S., Tyrolski, M., Kaiser, L., Wu, Y., Szegedy, C., and Michalewski, H. Hierarchical transformers are more efficient language models. In *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022. URL <https://aclanthology.org/2022.findings-naacl.117>.
- Nawrot, P., Chorowski, J., Lancucki, A., and Ponti, E. M. Efficient transformers with dynamic token pooling. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023. URL <https://aclanthology.org/2023.acl-long.353>.
- Neitemeier, P., Deiseroth, B., Eichenberg, C., and Balles, L. Hierarchical autoregressive transformers: Combining byte- and word-level processing for robust, adaptable language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tU074jg2vS>.
- OLMo Team, Walsh, P., Soldaini, L., Groeneveld, D., Lo, K., Arora, S., Bhagia, A., Gu, Y., Huang, S., Jordan, M., Lambert, N., Schwenk, D., Tafjord, O., Anderson, T., Atkinson, D., Brahman, F., Clark, C., Dasigi, P., Dziri, N., Guerquin, M., Ivison, H., Koh, P. W., Liu, J., Malik, S., Merrill, W., Miranda, L. J. V., Morrison, J., Murray, T., Nam, C., Pyatkin, V., Rangapur, A., Schmitz, M., Skjonsberg, S., Wadden, D., Wilhelm, C., Wilson, M., Zettlemoyer, L., Farhadi, A., Smith, N. A., and Hajishirzi, H. 2 olmo 2 furious. *arXiv preprint arXiv:2501.00656*, 2025.
- OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Owodunni, A. T., Ahia, O., and Kumar, S. Flexitokens: Flexible tokenization for evolving language models. *arXiv preprint arXiv:2507.12720*, 2025.
- Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J., Zettlemoyer, L., et al. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.
- Park, J. and Johnson, J. Rgb no more: Minimally-decoded jpeg vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 22334–22346, June 2023.
- Pérez, J. C., Pardo, A., Soldan, M., Itani, H., Leon-Alcazar, J., and Ghanem, B. Compressed-language models for understanding compressed file formats: a jpeg exploration. *arXiv preprint arXiv:2405.17146*, 2024.
- Petrov, A., Malfa, E. L., Torr, P., and Bibi, A. Language model tokenizers introduce unfairness between languages. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=78yDLKi95p>.
- Phan, B., Havasi, M., Muckley, M., and Ullrich, K. Understanding and mitigating tokenization bias in language models. *arXiv preprint arXiv:2406.16829*, 2024.

- Qu, X., Wang, S., Huang, Z., Hua, K., Yin, F., Zhu, R.-J., Zhou, J., Min, Q., Wang, Z., Li, Y., et al. Dynamic large concept models: Latent reasoning in an adaptive semantic space. *arXiv preprint arXiv:2512.24617*, 2025.
- Radford, A., Jozefowicz, R., and Sutskever, I. Learning to generate reviews and discovering sentiment. *arXiv preprint arXiv:1704.01444*, 2017.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners, 2019.
- Rae, J. W., Potapenko, A., Jayakumar, S. M., Hillier, C., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SylKikSYDH>.
- Rumbelow, J. and Watkins, M. Solidgoldmagikarp (plus, prompt generation), 2023.
- Rust, P., Lotz, J. F., Bugliarello, E., Salesky, E., de Lhoneux, M., and Elliott, D. Language modelling with pixels. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=FkSp8VW8RjH>.
- Salesky, E., Etter, D., and Post, M. Robust open-vocabulary translation from visual text representations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021. URL <https://aclanthology.org/2021.emnlp-main.576>.
- Salesky, E., Verma, N., Koehn, P., and Post, M. Multilingual pixel representations for translation and effective cross-lingual transfer. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.854>.
- Schmidt, C. W., Reddy, V., Zhang, H., Alameddine, A., Uzan, O., Pinter, Y., and Tanner, C. Tokenization is more than compression. *arXiv preprint arXiv:2402.18376*, 2024.
- Schmidt, C. W., Reddy, V., Tanner, C., and Pinter, Y. Boundless byte pair encoding: Breaking the pre-tokenization barrier. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=oPAjXGV8qQ>.
- Schuster, M. and Nakajima, K. Japanese and korean voice search. In *2012 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, 2012.
- Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016. URL <https://aclanthology.org/P16-1162>.
- Slagle, K. Spacebyte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*, 2024.
- Stern, M., Shazeer, N., and Uszkoreit, J. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 2018.
- Sun, L., Hashimoto, K., Yin, W., Asai, A., Li, J., Yu, P., and Xiong, C. Adv-bert: Bert is not robust on misspellings! generating nature adversarial samples on bert. *arXiv preprint arXiv:2003.04985*, 2020.
- Sutskever, I., Martens, J., and Hinton, G. E. Generating text with recurrent neural networks. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pp. 1017–1024, 2011.
- Tai, Y., Liao, X., Suglia, A., and Vergari, A. Pixar: Auto-regressive language modeling in pixel space. *arXiv preprint arXiv:2401.03321*, 2024.
- Tao, C., Liu, Q., Dou, L., Muennighoff, N., Wan, Z., Luo, P., Lin, M., and Wong, N. Scaling laws with vocabulary: Larger models deserve larger vocabularies. In *Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=sKCKPr8cRL>.
- Tay, Y., Tran, V. Q., Ruder, S., Gupta, J., Chung, H. W., Bahri, D., Qin, Z., Baumgartner, S., Yu, C., and Metzler, D. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=JtBRnr1OEFN>.
- Team, T. A. L. A. No more retokenization drift: Returning token ids via the openai compatible api matters in agent rl, 2025. URL <https://blog.vllm.ai/2025/10/22/agent-lightning.html>.
- Videau, M., Idrissi, B. Y., Leite, A., Schoenauer, M., Teytaud, O., and Lopez-Paz, D. From bytes to ideas: Language modeling with autoregressive u-nets. *arXiv preprint arXiv:2506.14761*, 2025.
- Vieira, T., LeBrun, B., Giulianelli, M., Gastaldi, J. L., DuSell, B., Terilla, J., O’Donnell, T. J., and Cotterell, R. From language models over tokens to language models over characters. *arXiv preprint arXiv:2412.03719*, 2024.

- Wang, D., Li, Y., Jiang, J., Ding, Z., Jiang, G., Liang, J., and Yang, D. Tokenization matters! degrading large language models through challenging their tokenization. *arXiv preprint arXiv:2405.17067*, 2024a.
- Wang, J., Gangavarapu, T., Yan, J. N., and Rush, A. M. Mambabyte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*, 2024b.
- Wang, S., Li, Z., Qian, H., Yang, C., Wang, Z., Shang, M., Kumar, V., Tan, S., Ray, B., Bhatia, P., Nallapati, R., Ramanathan, M. K., Roth, D., and Xiang, B. ReCode: Robustness evaluation of code generation models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023. URL <https://aclanthology.org/2023.acl-long.773>.
- Wei, H., Sun, Y., and Li, Y. Deepseek-ocr: Contexts optical compression. *arXiv preprint arXiv:2510.18234*, 2025.
- Wortsman, M., Liu, P. J., Xiao, L., Everett, K., Alemi, A., Adlam, B., Co-Reyes, J. D., Gur, I., Kumar, A., Novak, R., et al. Small-scale proxies for large-scale transformer training instabilities. *arXiv preprint arXiv:2309.14322*, 2023.
- Wu, S., Tan, X., Wang, Z., Wang, R., Li, X., and Sun, M. Beyond language models: Byte models are digital world simulators. *arXiv preprint arXiv:2402.19155*, 2024.
- Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., and Raffel, C. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. URL <https://aclanthology.org/2022.tacl-1.17>.
- Xuyang, S., Luo, X., Cheng, T., Chu, Z., Li, H., Huang, S., Zhu, Q., Wang, Q., Zhang, X., Zhou, S., et al. Is compression really linear with code intelligence? *arXiv preprint arXiv:2505.11441*, 2025.
- Yang, J. Rethinking tokenization: Crafting better tokenizers for large language models. *International Journal of Chinese Linguistics*, 2024.
- Yang, J., Wang, Z., Lin, Y., and Zhao, Z. Problematic tokens: Tokenizer bias in large language models. *arXiv preprint arXiv:2406.11214*, 2024.
- Yu, D., Cohen, E., Ghazi, B., Huang, Y., Kamath, P., Kumar, R., Liu, D., and Zhang, C. Scaling embedding layers in language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=gH4BRa4ZP3>.
- Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., and Lewis, M. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=JTmO2V9Xpz>.
- Zheng, B. S., Liu, A., Ahia, O., Hayase, J., Choi, Y., and Smith, N. A. Broken tokens? your language model can secretly handle non-canonical tokenizations. *arXiv preprint arXiv:2506.19004*, 2025a.
- Zheng, L., Yuan, J., Wang, C., and Kong, L. Efficient attention via control variates. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=G-uNfHKrj46>.
- Zheng, L., Zhao, X., Wang, G., Wu, C., Dong, D., Wang, A., Wang, M., Du, Y., Bo, H., Sharma, A., Li, B., Zhang, K., Hu, C., Thakker, U., and Kong, L. Evabyte: Efficient byte-level language models at scale, 2025b. URL <https://hkunlp.github.io/blog/2025/evabyte>.
- Zhu, T., Liu, Q., Wang, H., Chen, S., Gu, X., Pang, T., and Kan, M.-Y. Skyladder: Better and faster pretraining via context window scheduling. *arXiv preprint arXiv:2503.15450*, 2025.
- Zouhar, V., Meister, C., Gastaldi, J., Du, L., Sachan, M., and Cotterell, R. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023. URL <https://aclanthology.org/2023.acl-long.284/>.

Appendix

A. Token-level Compression Ratios for Proxy Compression

We implement mixed-representation training at the sample level. This indicates that although documents will be compressed with a certain probability r , the actual ratio of compressed tokens present in the context will differ from the sample-level mixing rate. In this section, we provide a rough estimate of the *token-level* compression ratio.

For simplicity, we consider a fixed batch size per training step, that is, the total number of symbols is fixed, denoted by M . For raw byte models, we assume each batch consists of N_b input documents each with average sample length L_b ; for a compression-based model, this comprises N_t input samples with average length L_t . We also denote the compression rate brought by compression as $C = \frac{L_b}{L_t}$. As a result, we have

$$M = N_t L_t = N_b L_b = N_b C L_t \rightarrow N_t = C N_b,$$

which indicates that compression-based models will approximately consume $C \times$ more samples than raw byte models. Now consider our proxy compression framework, where we additionally maintain a mixing ratio r such that with probability r an input sample will be compressed with a compressor at rate C . Denoting the total number of input samples (either compressed or not) per training step as N' , we have

$$M = r N' L_t + (1 - r) N' L_b = r N' \frac{L_b}{C} + (1 - r) N' L_b = \left[\frac{r}{C} + (1 - r) \right] N' L_b,$$

and by comparing the above, we obtain

$$N' = \frac{1}{\frac{r}{C} + (1 - r)} N_b,$$

which roughly estimates the number of input samples consumed in our compressed-raw mixture. For instance, for tokenizer-based proxies at a rate $r = 0.9$ with the OpenCoder tokenizer (Huang et al., 2024a), which delivers a compression rate $C \approx 3.7$, the actual token-level compression rate would be $\frac{1}{\frac{r}{C} + (1 - r)} \approx 2.91$.

Mixing with In-context Translation Pairing. When we employ translation pairs into training (§2.1), the actual compression rate is further decreased, as the same input sample will be duplicated twice. Since for each translation pair, it increments the amount of both compressed and raw data, we calibrate the mixing rate to take this into account. In particular, with the mixing rate r , we assume there will be rN samples compressed, while $(1 - r)N$ samples including both the raw and compressed versions. So the actual compression and raw samples would be $r + 1 - r : 1 - r$.

If $r > 0.5$, we mix translation pairs with compressed samples; conversely, if $r \leq 0.5$, we mix translation pairs with raw samples, as it would be impossible to attain a compressed-raw fraction larger than 0.5 if we solely mix translation and raw bytes. Concretely, if we desire a compressed fraction r , then the calibrated mixing rate r' of mixing translation pairs with probability r' must satisfy

$$\begin{cases} \frac{r' + (1 - r')}{r'} = \frac{r}{1 - r}, & \text{if } r > 0.5, \\ \frac{r'}{r' + (1 - r')} = \frac{r}{1 - r}, & \text{if } r \leq 0.5. \end{cases}$$

This gives

$$r' = \begin{cases} \frac{1}{r} - 1, & r > 0.5, \\ \frac{r}{1 - r}, & r \leq 0.5. \end{cases}$$

As a result, we can draw training examples with translation pairs appearing with probability r' , mixed with compressed (resp. raw) samples, where the effective fraction of compressed versus raw samples matches the intended target $r > 0.5$ (resp. $r \leq 0.5$).

We now investigate the delivered token-level compression rate with translation pairs. Assume $r > 0.5$ which implies $r' = \frac{1}{r} - 1$. Then for each training batch, we have

$$M = (1 - r')N'L_t + r'N'(L_b + L_t) = (1 - r')N'\frac{L_b}{C} + r'N'(L_b + \frac{L_b}{C}) = \left[\frac{1}{C} + r'\right] N'L_b.$$

This gives

$$N' = \frac{1}{\frac{1}{C} + \frac{1}{r} - 1} N_b,$$

with the configuration above, this yields an actual compression rate of $N' \approx 2.62N_b$, which is a bit slower than that without translation. As a result, we only employ translation pairing as a warmup phase at the beginning of training and disable it later to maximize the amount of unique data per training step.

In-context Translation Pairing Warmup. Also notice that when warmup phases are incorporated, we consider annealing $r(i)$ linearly from the start point a and ending rate b with T steps (in this work, $a = 0.4$ and $b = 0.9$ by default). For $r \leq 0.5$, we can follow similar reasoning and derive

$$M = (1 - r'(i))N'_i L_b + r'(i)N'_i(L_b + L_t) = (1 - r'(i))N'_i L_b + r'(i)N'_i(L_b + \frac{L_b}{C}) = \left[1 + \frac{r'(i)}{C}\right] N'_i L_b.$$

This gives

$$N'_i = \frac{1}{1 + \frac{r'(i)}{C}} N_b = \frac{1}{1 + \frac{r(i)}{1-r(i)} \frac{1}{C}} N_b.$$

The compression rate for $r > 0.5$ can be calculated similarly. We evaluate the average number of samples processed at each step as $\bar{N}' := \frac{1}{T} \sum_{i=1}^T N'_i$. This can be evaluated either by enumerating each term in the summation, or by approximating with the piecewise integral average, which can be evaluated in closed form to give a quick estimate. For instance, in our default schedule, we anneal $r(i) = 0.4 + \frac{0.5i}{10000}$ to increase from 0.4 to 0.9 at the first 10000 training steps i . This gives the average rate $\bar{N}' \approx 1.38N_b$, in other words, during the warmup phase, we only process on average $1.38\times$ as many samples per step as a pure raw byte model training step.

B. Implementation Details of Proxy Compressors

B.1. Tokenizer-based Proxy Compression

This section provides additional implementation details on tokenizer-based proxy compression, including representation formats, encoding strategies, and pre-tokenization schemes. Table 5 summarizes ablation results across these design choices.

Representation Formats. We consider two formats for representing tokenizer outputs:

- **Token indices** (default): Each token ID is mapped to a dedicated entry in the language model vocabulary and treated as a single symbol. This achieves the highest compression rate ($3.7\times$ with the OpenCoder tokenizer) and yields the best downstream performance.
- **Token bytes**: Token IDs are serialized as fixed-length byte sequences. We choose the smallest B such that $256^B \geq V$, where V is the tokenizer vocabulary size. For OpenCoder ($V = 96,640$), this gives $B = 3$, so each token becomes 3 bytes. Under this scheme, a length- L token sequence produced by the tokenizer is converted into a sequence of BL symbols. This format keeps both raw and compressed representations in byte space, which we hypothesized might ease cross-representation alignment. However, it reduces the effective compression rate and does not improve transfer (Table 5).

A practical advantage of token bytes is flexibility: since the language model vocabulary is decoupled from the tokenizer vocabulary, arbitrarily large tokenizers (including “superword” vocabularies that merge across word boundaries (Liu et al., 2025)) can be supported without changing the model’s embedding table. For our token-byte experiments, we train a BPE tokenizer with a default vocabulary size $V = 65,536$ on our pretraining corpus, so that each token ID can be represented as exactly 2 bytes.

Table 5. Ablation study on design choices of tokenizer proxy compression. All models are 1.5B parameters trained on 100B sequence symbols from the Python subset. Direct token-index representation achieves the best compression-performance trade-off.

Format	Vocab Size	Pre-tokenization	Encoding	Compression Rate	Pass@1 (%)
Byte	256	Default	Default	1.7	14.6
Byte	256	Default	Gray Coding	1.7	15.2
Byte	256	Default	Huffman Coding	1.8	18.3
Double-byte	65536	Default	Default	3.5	15.2
Byte	256	Line	Default	2.3	18.3
Byte	256	SuperBPE	Default	2.9	16.5
Token-index	96640	Default	—	3.7	20.7

We also explore a *double-byte* variant that expands the language model vocabulary from 256 to 65,536. Under this scheme, each token ID maps to a single symbol, rather than being serialized into multiple bytes. As shown in Table 5, double-byte achieves a higher compression rate ($3.5\times$) than simple bytes ($1.7\times$) but still underperforms direct token-index representations.

Byte Encoding Strategies. For the token-bytes format, we explored several encoding strategies:

- **Default (fixed-length):** Each token ID is converted to a B -byte big-endian representation. All tokens occupy the same number of bytes, providing uniform representation complexity. Note that this implicitly encodes frequency information, since BPE typically assigns lower IDs to more frequent tokens.
- **Huffman coding:** Variable-length byte codes are assigned based on token frequency, with more frequent tokens receiving shorter codes. Codes are byte-aligned and satisfy the Kraft-McMillan inequality for prefix-free decodability. In practice, this yields only marginal compression improvements over fixed-length encoding.
- **Gray coding:** Tokens are first sorted lexicographically by their UTF-8 surface forms and assigned sequential ranks. These ranks are then converted to Gray codes, ensuring that lexicographically similar tokens differ by only one bit in their byte representations. This preserves locality in byte surfaces and slightly improves performance.

As shown in Table 5, these encoding strategies yield marginal improvements in downstream performance within the token-bytes format, and all underperform direct token-index representations.

Pretokenization Schemes. Standard BPE pipelines first segment raw text into “words” via pre-tokenization (typically regex-based), then apply iterative merges to produce subwords. We experimented with several pre-tokenization schemes:

- **Default:** Standard regex-based pre-tokenization (Radford et al., 2019; OpenAI, 2023; Dagan et al., 2024).
- **Line-separated:** Pre-tokenization allows tokens to span multiple words within a line (Fried et al., 2023).
- **SuperBPE:** Extended pre-tokenization that permits merges across whitespace boundaries (Liu et al., 2025).

Line-separated and SuperBPE pre-tokenization improve compression rates ($2.3\times$ and $2.9\times$ respectively, compared to $1.7\times$ for default), but do not match the performance of direct token-index representations.

Tokenizer Algorithms. We also compared Unigram-based tokenization (Kudo, 2018) and found no significant difference in transfer performance at matched compression rates, suggesting that the choice of tokenization algorithms is less critical.

B.2. Neural Proxy Compression

This section provides implementation details for neural proxy compression, including the compressor architecture, entropy-based segmentation algorithm, and parallelization strategies. Table 6 summarizes ablation results across key design choices.

Compressor Model. Our neural proxy compressor combines a small byte-level language model with arithmetic coding. We train a $\sim 40\text{M}$ -parameter byte-level Transformer on our pretraining corpus, with the dimension size 512, 12 layers,

8 attention heads. Training uses a learning rate of $1e-3$ for 200k steps with batch size 64×2048 bytes. The model operates over a fixed alphabet of size 256, and produces conditional distributions $p(b_t \mid b_{<t})$ for every byte in the input $x_{\text{raw}} = (b_1, \dots, b_T)$. These distributions drive the arithmetic coder to output a compressed bitstream.

Entropy-based Segmentation. Naively compressing training data is prohibitively slow: (1) the arithmetic coder is inherently sequential, and (2) the equal-information window scheme in [Lester et al. \(2024\)](#) requires frequent termination and context truncation, triggering new forward passes for the language model to obtain new probabilities. To increase parallelism, we pre-segment training samples and compress each segment independently.

We implement an *entropy-based adaptive segmentation* strategy: for each input example in UTF-8 bytes, we run the model forward pass to obtain per-position logits and compute the per-byte cross-entropy $h_t := -\log p_t(b_t)$ for $t = 1, \dots, L$. Segment boundaries are constructed where the entropy profile indicates low predictability based on two criteria:

1. **Absolute entropy:** positions where h_t exceeds a global threshold (e.g., 95th percentile).
2. **Entropy jumps:** positions where the finite difference $\Delta h_t = h_t - h_{t-1}$ exceeds a monotonicity threshold, indicating sudden changes in predictability.

Similar entropy-based criteria also appear in BLTs ([Pagnoni et al., 2024](#)) for dynamic byte patchification within the model architecture; here we use them only for segmenting inputs for parallel arithmetic coding. By default, we use the 95th percentile as the threshold for both criteria.

Vectorized Arithmetic Coding. We implement a batched arithmetic coder to process multiple segments in parallel with equal-information windows ([Lester et al., 2024](#)). Given a predefined bit threshold τ , compression proceeds iteratively for each segment:

1. Run a forward pass of the compressor model to obtain next-byte distributions (on GPU).
2. Perform arithmetic coding and count output bits in the resulting compressed bitstream (on CPU). If the bitstream for the current window exceeds τ bits, emit the first τ bits, discard the consumed byte context, and return to step 1 with the truncated context.

We design a pipelined implementation to overlap GPU forward passes with CPU encoding across iterations.

Caching. We exploit substantial repetition in pre-training corpus with a global segment cache: before compressing a segment, we check whether it exists in the cache. Cache hits will retrieve the pre-computed compressed bitstream, while cache misses will trigger compression as usual. Caching significantly reduces redundant computation during arithmetic coding.

Bitstream Packing. After compressing each segment to a bitstream, we pack fixed-length bit chunks into discrete symbols. We define a bitwidth N and pack consecutive N -bit chunks from the bitstream into symbols. By default, $N = 16$, which yields a vocabulary size $V = 65,536$ for the language model embedding table; we ablate this design choice in [Table 6](#).

End-to-End Pipeline. The compressor runs fully offline in parallel via a multi-process pipeline. Each worker:

1. Reads a shard of the corpus.
2. Applies entropy-based segmentation (on GPUs).
3. Compresses segments with arithmetic coding, equal-info windows ([Lester et al., 2024](#)), and cache lookup (GPU/CPU pipelining).
4. Packs the resulting compressed bitstream into fixed-bit symbols.
5. Writes segmentation metadata and compressed sequences.

At training time, the proxy compressor simply reads the pre-computed compressed data and presents them to the mixed-representation training pipeline ([§2.1](#)). Our pipeline design improves efficiency significantly: we process $\sim 3.3\text{TB}$ of pretraining data at 0.57 GB/hour per process, compared to 0.005 GB/hour for a naive fully-sequential implementation.

Table 6. Ablation study on design choices of neural proxy compression with 1.5B models. Compressor CR denotes the compression rate of the proxy compressor in isolation (i.e., without mixing with raw bytes). Pre-segmentation controls how documents are split before arithmetic coding; EqualInfoAC Bits specifies the target information per coding window (Lester et al., 2024); Packing Bits determines how many bits are grouped into each discrete symbol (V is the resulting vocabulary size). The bottom row shows our final configuration, which achieves an effective compression rate of approximately 2.6 under the mixed-representation training scheme with $r=0.9$ (see Appendix A).

Pre-segmentation	EqualInfoAC Bits	Packing Bits	Compressor CR	Pass@1 (%)
Fixed-length (size=4)	None	8 ($V=256$)	1.4	8.5
Fixed-length (size=8)	None	8 ($V=256$)	2.0	3.7
Fixed-length (size=16)	None	8 ($V=256$)	2.9	6.1
Lines	None	8 ($V=256$)	5.1	9.8
Lines	16	8 ($V=256$)	2.3	11.6
Entropy (98th percentile)	16	8 ($V=256$)	1.8	13.4
Entropy (98th percentile)	16	16 ($V=65536$)	3.5	18.9
Entropy (95th percentile)	32	16 ($V=65536$)	4.1	12.2
Entropy (95th percentile)	16	16 ($V=65536$)	3.1	18.3

Why Neural Compression Cannot Decode. Unlike tokenization, neural compression is not invertible and thus does not support decoding. Several factors contribute to this: (i) probability tables are not stored during compression, (ii) our use of segmentation and equal-information windowing (Lester et al., 2024) produces compressed segments of varying byte lengths, and these boundaries are unknown at decode time, and (iii) at inference time, when new compressed symbols are predicted by the language model instead of produced by the compressor, the underlying compressor probabilities that would be needed for decoding are by definition unavailable. These factors cause the same compressed sequence to correspond to multiple plausible raw byte sequences, or the “fuzziness” discussed in §2.3.

Design Choice Ablations. Table 6 ablates key design choices in our neural compression: segmentation strategy, equal-information bit window size, and symbol packing granularity. For *segmentation*, we observe fixed-length segmentation performs poorly, likely due to abrupt changes with semantic-agnostic boundaries. Line-based segmentation (splitting at newlines) works well for code, improves results, but remains unsatisfactory. Entropy-based segmentation is general and yields the best performance by placing boundaries at natural transition points. When using equal-info windows (Lester et al., 2024), it improves performance but degrades compression rates, as window resets introduce overhead in terms of compression. We find 16-bit windows strike the best balance and increasing to 32 bits improves compression rate but greatly degrades performance. For packing, using 16-bit per symbol substantially outperforms the 8-bit variant. In addition, the choice between 95th and 98th percentile for the threshold in entropy segmentation has minimal impact on downstream performance. Overall, entropy-based segmentation with 16-bit equal-information windows and 16-bit symbol packing yields the best trade-off.

C. Additional Architectural, Training, and Evaluation Details

C.1. Model Architectures

Figure 10 compares EvaByte (Zheng et al., 2025b) against OpenCoder (Huang et al., 2024a), a Llama-based transformer architecture (Dubey et al., 2024). We reproduce OpenCoder 1.5B and 8B under our codebase; our reproductions in a longer training run match or exceed the released intermediate checkpoints on HumanEval and MBPP benchmarks. EvaByte consistently achieves lower BPB while matching OpenCoder’s downstream performance at comparable model sizes, where both models operate on tokens. Given its favorable efficiency–performance trade-off (Zheng et al., 2023; 2025b), we adopt EvaByte as the default architecture throughout this study. Following EvaByte (Zheng et al., 2025b), we employ multi-symbol (over either tokens or bytes) prediction (Stern et al., 2018; Gloeckle et al., 2024; Cai et al., 2024; Zheng et al., 2025b; Grivas et al., 2025) as the training objective above 4B parameters, and standard next-symbol prediction otherwise (enabling multi-symbol prediction for smaller models leads to slightly degraded performance, consistent with Gloeckle et al. (2024)). Across different models, for byte-level predictions, we use 8 prediction heads; for token-level predictions from (proxy) compressors, we use either standard next-token prediction or 2-head multi-token prediction (we conducted ablation studies and found varying the number of token prediction heads does not improve downstream performance, including for our proxy

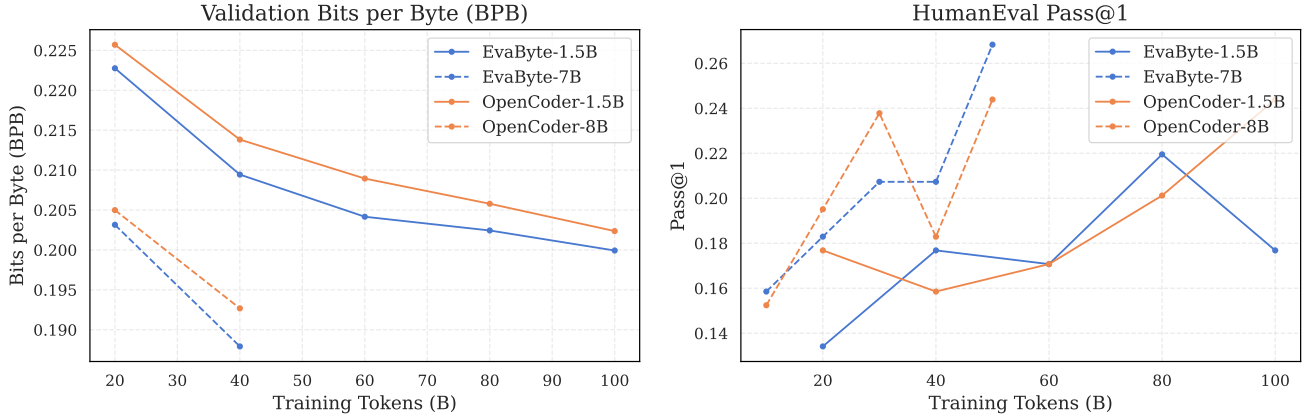


Figure 10. Ablation on model architecture on validation BPB (left) and HumanEval pass@1 (right).

Table 7. Architectural hyperparameters. Vocabulary sizes for EvaByte are omitted here due to varying input representations in our study.

Hyperparameter	OpenCoder		EvaByte				
	1.5B	8B	0.5B	1.5B	4B	7B	14B
Layers	24	32	28	20	32	32	40
Model Dimension	2240	4096	1024	2048	3072	4096	5120
FFN Dimension	6144	14336	4096	8192	9216	12288	16384
Attention Heads	14	32	8	16	24	32	40
Key / Value Heads	14	8	8	16	24	32	40
Vocab Size	96640	96640	-	-	-	-	-
RoPE θ	10000	500000	100000	100000	100000	100000	100000
Context Window Size	4096	4096	16384	16384	16384	16384	16384

models, which ultimately operate on raw bytes at inference). We choose the number of token- or byte-prediction heads to roughly match prediction granularity across representations, given that both tokenizer and neural proxy compressors achieve compression rates above $3\times$, although we found it does not lead to performance improvements over other configurations.

C.2. Training Configuration

Table 8. Learning rates for different input representations and model sizes. All values are peak learning rates of the training schedule.

Variant	0.5B	1.5B	4B	7B	14B
Tokenizer	$1.2e-3$	$9e-4$	$7e-4$	$5e-4$	$5e-4$
Byte-level	$1.2e-3$	$9e-4$	$7e-4$	$5e-4$	$3e-4$
Proxy (Neural)	$1.2e-3$	$9e-4$	$3e-4$	$5e-4$	$3e-4$
Proxy (Tokenizer)	$1.2e-3$	$9e-4$	$7e-4$	$5e-4$	$3e-4$

Model parameters are initialized from a truncated normal distribution with standard deviation 0.02, except for embedding parameters (with standard deviation 1.0). Gradient norms are clipped to 1.0. We use AdamW (Kingma & Ba, 2014; Loshchilov & Hutter, 2019) with weight decay 0.1, $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\epsilon = 1e-15$, following prior practices on stable large-scale training (Molybog et al., 2023; Wortsman et al., 2023; OLMo Team et al., 2025; Zheng et al., 2025b). Table 8 reports peak learning rates for different input representations and model sizes. For each configuration, we perform a moderate learning rate sweep while keeping the batch size fixed, starting from common values and adjusting until observing either training instability or validation loss degradation.

For models trained on the Python subset, we use a fixed effective batch size of 2M sequence *symbols*, independent of whether the inputs are represented as tokens, bytes, or other compressed formats. As a result, models with different input

Table 9. Downstream pass@10 on HumanEval-Plus across model sizes and input representations under the same training setup as Table 1.

Model	Model Size				
	0.5B	1.5B	4B	7B	14B
Tokenizer-based	24.3	28.3	38.4	43.4	44.2
Byte-level	22.9	27.5	32.0	32.7	35.3
Proxy (Neural)	21.6	26.5	33.6	42.1	44.5
Proxy (Tokenizer)	20.2	28.7	42.0	44.3	48.1

representations consume different amounts of raw data per batch. These models are trained for 50000 steps with a cosine learning-rate schedule, where the learning rate is linearly warmed up for the first 500 steps and then decayed down to 10% of its peak value. For models trained on the full GitHub corpus (Table 2), we increase the batch size to 4M sequence symbols for 80000 training steps and use a constant learning rate schedule with 2000-step linear warm-up. All other settings remain unchanged.

Packing and Document Boundaries. Stateful proxy compressors such as gzip and neural compression operate on full raw documents *before* packing; raw or compressed documents are then wrapped with format sentinels and packed into fixed-length contexts following standard practice. Thus, compressor state never crosses document boundaries. For neural compression, equal-information windows (Lester et al., 2024) additionally reset the compressor state at fixed bit budgets within a document (§2.3), limiting the range over which compression state is carried. If long documents are truncated during context packing, truncation only removes already-compressed suffixes. Document-boundary attention masking within packed contexts (Appendix D.6) further prevents cross-document attention.

C.3. Evaluation Protocols

We primarily focus on downstream code generation tasks for evaluation and report pass@ k rates following Chen et al. (2021), which estimate the probability of a model generating a correct solution within k attempts. We evaluate on well-established code generation benchmarks, including HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and their EvalPlus variants (Liu et al., 2023). Pass@1 is calculated via greedy decoding, and pass@10 draws 20 samples at temperature 0.2 using nucleus sampling (Holtzman et al., 2020) with top- $p = 0.95$. To accelerate decoding, we cap the maximum number of generated tokens to 512 for tokenizer-based baselines and 2048 for other input formats. We also measure *Bits-Per-Byte* (BPB) for representation-agnostic comparison of modeling quality (Rae et al., 2020; Xue et al., 2022; Yu et al., 2023; Huang et al., 2024b; Xuyang et al., 2025), computed on 40K held-out samples (~ 150 M bytes) from SwallowCode (Fujii et al., 2025).

Note on BPB Evaluation. We focus on downstream task performance rather than validation BPB in this work. Validation BPB is known to be biased when comparing across different data representations (Vieira et al., 2024; Hwang et al., 2025), and this bias is amplified in our setting where training involves tokens from different compressors, raw bytes, and their mixtures. Empirically, we observe that models trained with multi-symbol prediction objectives (whether over tokens or bytes) often achieve better downstream code generation but worse BPB, likely because their training objective diverges from next-symbol likelihood. Similarly, proxy-trained models exhibit worse BPB when evaluated on either compressed or raw representations alone, despite superior downstream performance. This is expected as at training time, the model must not only predict next symbols within each representation, but also align across representations to achieve transfer, which goes beyond standard next-symbol prediction. We therefore rely on downstream benchmarks as our primary metric, which we find yields reliable signals across models and scales.

D. Additional Experimental Results and Analyses

D.1. Full Results of Downstream Transfer

We list pass@1 rates on HumanEval and MBPP, as well as their EvalPlus variants for completeness, as shown in Tables 10 and 11. We also report HumanEval-Plus pass@10 rates in Table 9 and observe a consistent trend with pass@1 rates.

Table 10. Downstream pass@1 performance on HumanEval(-Plus) and MBPP(-Plus) across different model sizes and input representations.

Task	Model	Compression Rate	Model Size				
			0.5B	1.5B	4B	7B	14B
HumanEval	Tokenizer-based	3.7	21.3	21.3	33.5	32.9	34.1
	Byte-level	1.0	17.7	21.3	26.8	26.2	27.4
	Proxy (Neural)	2.6	15.2	21.3	26.2	30.5	32.9
	Proxy (Tokenizer)	2.9	13.4	23.8	29.3	32.9	34.8
HumanEval-Plus	Tokenizer-based	3.7	17.7	18.3	28.0	28.7	29.3
	Byte-level	1.0	15.9	18.3	22.0	23.8	24.4
	Proxy (Neural)	2.6	13.4	18.3	22.6	26.8	29.9
	Proxy (Tokenizer)	2.9	12.2	20.7	24.4	26.2	30.5
MBPP	Tokenizer-based	3.7	37.8	47.9	57.1	54.8	59.8
	Byte-level	1.0	31.7	42.1	51.1	50.0	52.6
	Proxy (Neural)	2.6	27.5	37.3	50.8	50.0	58.7
	Proxy (Tokenizer)	2.9	30.7	46.0	54.2	56.3	58.2
MBPP-Plus	Tokenizer-based	3.7	29.4	41.0	46.3	45.2	48.1
	Byte-level	1.0	25.9	33.6	41.8	41.3	42.1
	Proxy (Neural)	2.6	22.0	29.6	41.8	41.8	49.2
	Proxy (Tokenizer)	2.9	25.4	38.4	44.4	45.5	49.5

Table 11. Downstream pass@1 performance on HumanEval(-Plus) and MBPP(-Plus) after training 320B sequence symbols on the full RefineCode GitHub corpus.

# Parameters	Model	Compression Rate	HumanEval	HumanEval-Plus	MBPP	MBPP-Plus
1.5B	Tokenizer-based	3.7	19.5	17.1	33.9	28.0
	Byte-level	1.0	11.6	9.1	31.0	23.3
	Proxy (Neural)	2.6	15.2	14.0	29.4	24.1
	Proxy (Tokenizer)	2.9	14.6	12.8	32.8	25.1
7B	Tokenizer-based	3.7	25.6	21.3	42.6	36.0
	Byte-level	1.0	18.3	14.6	41.5	32.5
	Proxy (Neural)	2.6	25.0	21.3	45.2	36.0
	Proxy (Tokenizer)	2.9	23.2	22.0	43.4	34.9

D.2. Additional Analyses of In-context Transfer

Following §3.3, we provide additional experimental details and analyses for in-context transfer probing under controlled supervision. We consider the following mixed-representation prompt for evaluation,

$$[\langle \text{comp} \rangle \circ p_{\text{comp}} \circ s_{\text{comp}} \circ \langle / \text{comp} \rangle \circ \langle \text{raw} \rangle \circ p_{\text{raw}}].$$

We evaluate both tokenizer-based and neural compressors at 1.5B and 7B models, tracking oracle-translation pass@1 at intermediate training steps (10k, 20k, 30k, 40k, and 50k). As described in §2.1, we optionally transform training inputs into explicit translation pairs by including both compressed and raw views of the same data in the same context. To study how in-context transfer strength depends on this pairing schedule, we consider three variants that share the same overall ratio $r = 0.9$ of compressed samples and differ only in whether those samples are paired: (1) *No pairs*, where training examples are independently sampled into either compressed or raw form without any paired data; (2) *Warmup-only*, where the first 10k steps use translation pairs and later steps switch to independent sampling; and (3) *Always-on*, where translation pairs are present throughout training. We identify two key findings from the extended in-context transfer experiments.

Structural differences in proxy compression drive transfer stability. As shown in Table 3, under *Warmup-only*, the tokenizer proxy drops after warmup but partially recovers (1.5B: 90.9%→31.1%→45.7%), while neural proxy decays more severely (1.5B: 90.9%→14.6%→38.4%), reflecting the structural properties of the two proxy compression types (Figure 4). Tokenizer-based compression uses a global static vocabulary where token IDs map consistently to byte patterns across

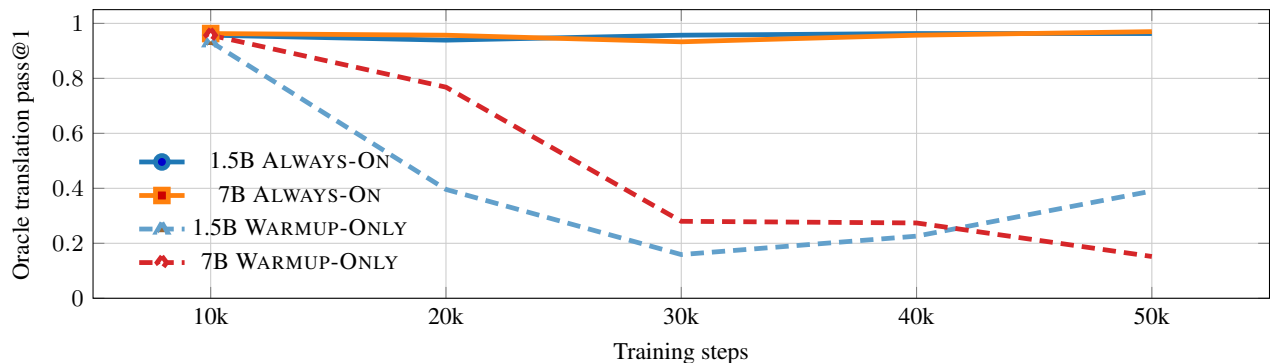


Figure 11. Oracle translation pass@1 on neural proxy compression. Solid lines represent continual translation pairs (ALWAYS-ON), while dashed lines indicate abrupt removal after 10k steps (WARMUP-ONLY).

the corpus, providing a stable anchor even without explicit supervision. Neural compression, in addition to its structured fuzziness (§3.4), produces context-dependent symbols that lack a fixed global mapping, where the same pattern may yield different compressed sequences depending on local context. This potentially explains why, under ALWAYS-ON, both proxy compressors reach very high translation pass@1 (the model can always rely on the explicit translation pairing); whereas under WARMUP-ONLY, once explicit pairs disappear, neural proxies decay faster than tokenizer-based counterparts in oracle-translation pass@1.

Larger models do not substitute for explicit pairing. Figure 11 visualizes the training dynamics for neural compression at 1.5B and 7B models. Under *Warmup-only*, both 1.5B and 7B models exhibit rapid decay in oracle-translation accuracy once translation pairs are removed, despite 7B retaining slightly higher accuracy at intermediate steps (e.g., 73.8% vs. 39.0% at 20k). This suggests that in-context translation is fundamentally dependent on explicit pairing supervision: larger models do not internalize the superficial mapping more robustly, they simply start from a marginally better initial alignment. Once pairing is removed, the language modeling objective no longer constrains surface-level translation, and both scales converge toward similar degraded performance. In contrast, under *Always-on*, both 1.5B and 7B maintain near-perfect translation (>94%) throughout training, confirming that a small fraction of explicit pairs is both necessary and sufficient for reliable in-context transfer.

D.3. Additional Results of Neural Proxy Compression

We provide additional analysis of the collision behavior induced by the neural compressor, complementing the main results in §3.4.

LCP Ratio Distribution. To quantify similarity among colliding chunks, we compute the *longest common prefix (LCP) ratio*: the length of the shared prefix divided by average chunk length. Figure 12 shows that over 90% of collisions have LCP ratios above 0.8, meaning colliding chunks are nearly identical except for short suffixes. This confirms that neural compression induces structured rather than arbitrary ambiguity.

Representative Collision Cases. Table 12 presents statistics for four representative collision clusters, and Listing 1 shows qualitative examples. Even in the largest cluster (Case 1, 61 variants), nearly all variants differ only by trailing whitespace and indentation. Cases 5–7 illustrate common patterns: URL suffixes, `if __name__` boilerplate with varying completions, and function calls with formatting differences. These examples demonstrate that neural compression merges semantically equivalent content while abstracting away superficial formatting noise.

Listing 1. Collision examples from the neural compressor.

Case 1: Maximum scale (61 variants, 4 shown)

```
[, \n
                                ]
[, \n
                                ]
[, \n
                                ]
[, \n
                                ]
```

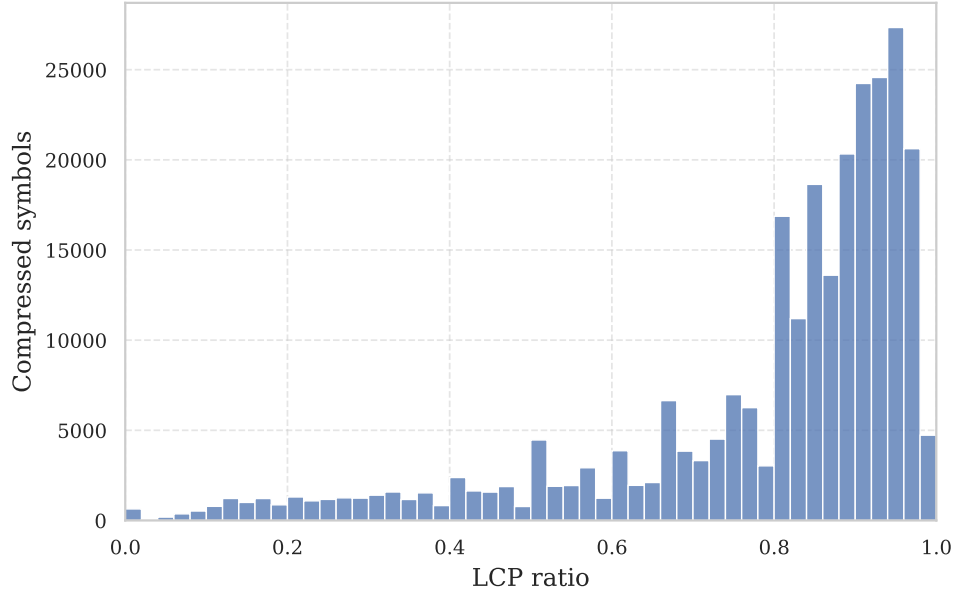


Figure 12. Distribution of the LCP ratio across all compressed symbols that exhibit collisions. The distribution is heavily skewed towards high similarity (LCP ratio > 0.8), showing that collisions typically differ only in short suffixes.

Table 12. Statistics of four representative collision clusters induced by the neural compressor.

Case	# Variants	LCP Ratio	Length (mean)	Std
1. Maximum scale	61	0.49	59.00	17.60
2. Minimum scale	2	0.56	64.00	28.00
3. Highest LCP	2	0.87	7.50	0.50
4. Lowest LCP	4	0.00	10.00	8.15

Case 2: Minimum scale (2 variants)

```
[acc,\n
[acc,\n
]
```

Case 3: Highest LCP ratio (2 variants)

```
[True\n\n ]
[True\n\n ]
```

Case 4: Lowest LCP ratio (4 variants)

```
[ ]
[ ]\n
[!]
[ ]\n
]
```

Case 5: URL suffixes (4 variants)

```
[ = 1)\n\n# + colab={}]
[ = 1)\n\n# + colab={"base_uri": "https://localhost:8080/"}
[ = 1)\n\n# + colab={"base_uri": "https://localhost:8080/", "height":  }
[ = 1)\n\n# + colab]
```

Case 6: Main function boilerplate (11 variants)

```
[\nif __name__ == '__']
[\nif __n]
[\nif __name__ == '__main]
[\nif __name__ == '__main__']
[\nif __]
[\nif __]
```

```
[\\nif __name__ ]
[\\nif __name__ =]
[\\nif __name__ == ' __]
[\\nif __na]
[\\nif __name__]
```

Case 7: Function call formatting (4 variants)

```
[process()\\n
[process(\\n
[process()\\n
[process(\\n ]]
```

D.4. Additional Results of Robustness Evaluation on ReCode

Evaluation Metrics. We evaluate robustness on the HumanEval split of the ReCode benchmark (Wang et al., 2023), which applies naturally occurring, semantics-preserving perturbations to coding problems. Following ReCode (Wang et al., 2023), we consider four perturbation families: function name rewrites (*Function*), formatting changes (*Format*), syntactic rewrites (*Syntax*), and docstring paraphrases (*Docstrings*). We report the *nominal* pass rate where no perturbations are applied. We measure robustness based on three metrics: *Robust Pass* $RP_s@k$, measuring worst-case pass@ k under s variants of the same problem. Using the standard pass@ k estimator with n samples, robust pass rates are defined as $RP_s@k = \mathbb{E}_x[1 - \binom{n-r_c^s(x)}{k} / \binom{n}{k}]$, where $r_c^s(x)$ counts generations that pass *all* s variants (Wang et al., 2023). Intuitively, RP measures the worst-case pass rates where a problem is solved only when the solution remains correct no matter how the prompt is perturbed (higher is better). *Robust Drop* $RD_s@k = (\text{pass}@k - RP_s@k) / \text{pass}@k$ measures the relative degradation from nominal to worst-case performance (lower is better; negative values indicate gains under perturbations). *Robust Relative (flip rate)* $RR_s@k$ captures stability: the proportion of samples whose correctness flips between the nominal and perturbed inputs (lower is better).

For each original problem in HumanEval (Chen et al., 2021), ReCode provides $s = 5$ randomly perturbed variants for each perturbation type. We run greedy decoding with $n = 1$ sample and $k = 1$. We thus refer to the resulting quantities simply as RP, RD, and RR.

Models. We compare our proxy-trained models against the tokenizer-based and byte-level baselines at 7B parameters, using the same checkpoints as in Table 1. To evaluate robustness more extensively, we additionally evaluate tokenizer-proxy-trained models by running inference on tokens, denoted by **Proxy (Tokenizer-tokens)**.

Per-Family Analysis. We visualize RP, RD, and RR for each perturbation family. *Format perturbations* (Figure 13) exhibit the largest gap between representations. The tokenizer baseline suffers severe degradation, while byte-level and proxy models remain stable or even improve. This indicates that partial exposure to raw bytes during training suffices to improve model robustness against surface-level formatting noise. We also visualize results for *Function perturbations* (Figure 14), *Syntax perturbations* (Figure 15), and *Docstring perturbations* (Figure 16).

D.5. Additional Analyses on Data Representations

While tokenization remains the dominant approach for transforming raw inputs into discrete units for language models, we systematically investigate a spectrum of input representations ranging from BPE tokens down to individual bits. Specifically, we consider BPE tokens, double-bytes (16-bit), bytes (8-bit), half-bytes (4-bit), double-bits (2-bit), and bits (1-bit), training models with matched parameter counts and training FLOPs across all representations.

Figure 17 reveals two complementary trends. Under a fixed compute budget, representations that process more data per FLOP achieve lower validation BPB, indicating superior *compute efficiency*. Conversely, under a fixed data budget (right), lower-level representations generally outperform higher-level ones, reflecting better *data efficiency*, consistent with prior findings on byte-level models (Xue et al., 2022; Zheng et al., 2025b). However, this trend does not extrapolate to the finest granularities: 2-bit and 1-bit models consistently underperform across both regimes. This suggests that excessively fine-grained representations waste compute greatly and lack sufficient abstraction for effective learning, even when granted additional training steps or data. We observe consistent trends in longer training runs (Appendix D.7). These findings motivate representations that balance abstraction with granularity, rather than pursuing either extreme.

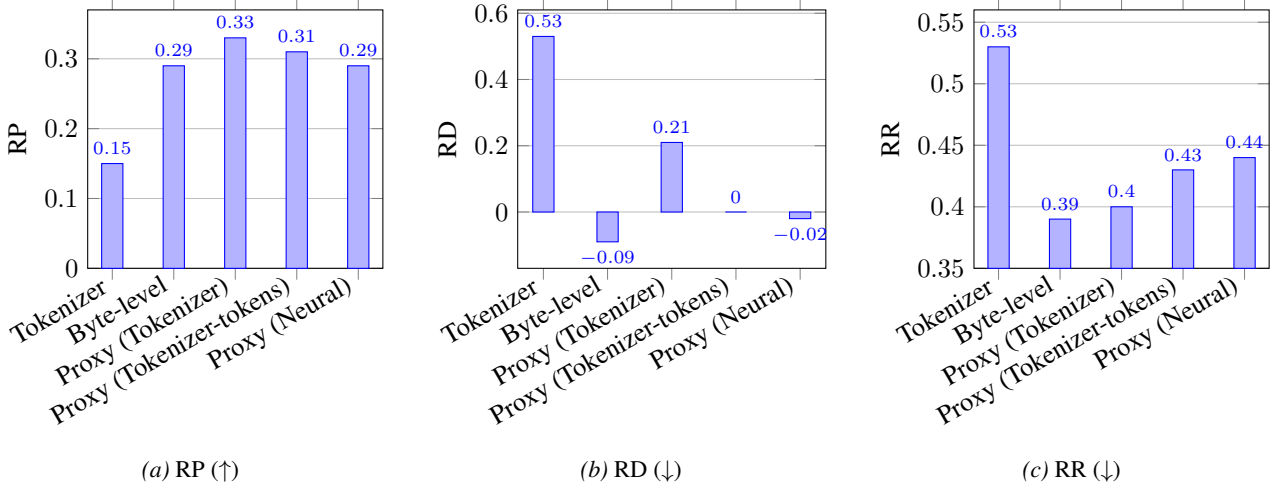


Figure 13. Format perturbations.

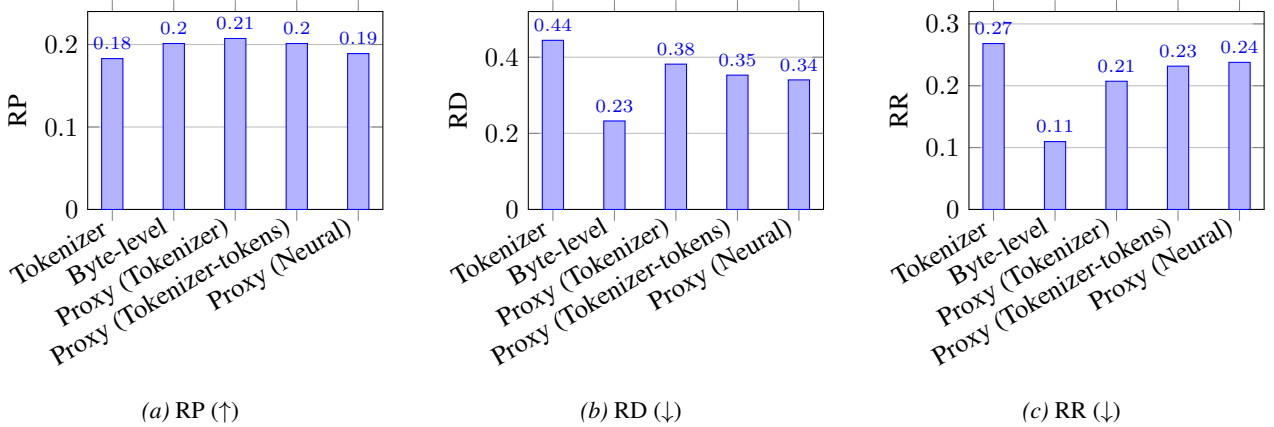


Figure 14. Function perturbations.

D.6. On Document-boundary Attention Masking

In proxy compression training, multiple samples potentially under different representations are concatenated and packed into fixed-length contexts, following standard language model training practice. This raises a natural question: does cross-representation transfer arise from in-context attention between samples of different representations, or from shared model parameters?

To isolate this, we compare two settings: (1) standard packing, where samples within a context can attend to each other, and (2) document-boundary attention masking, which restricts attention to within-document tokens only. As shown in Figure 18, we observe performance improvements when preventing cross-document attention. This suggests that cross-representation transfer stems primarily from shared parameters rather than in-context interactions between representations. Following recent work demonstrating benefits of document-boundary masking for training (Zhu et al., 2025), we enable it by default in all other experiments unless otherwise specified.

D.7. Additional Results on Data Efficiency versus Compute Efficiency

In the main text (§3.2), we showed that proxy compression captures the best of both data efficiency and compute efficiency for 14B models (Figure 3). Here we provide extended analysis across model scales and training horizons.

We note that intermediate checkpoint comparisons can be partially confounded by learning-rate annealing; models compared at the same amount of consumed data can correspond to different training steps and thus different points on the learning-rate

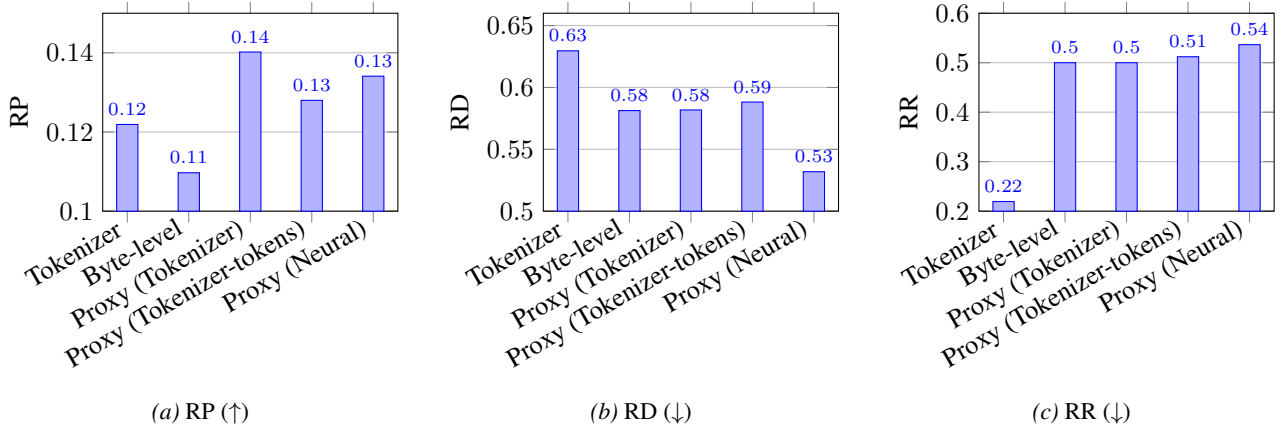


Figure 15. Syntax perturbations.

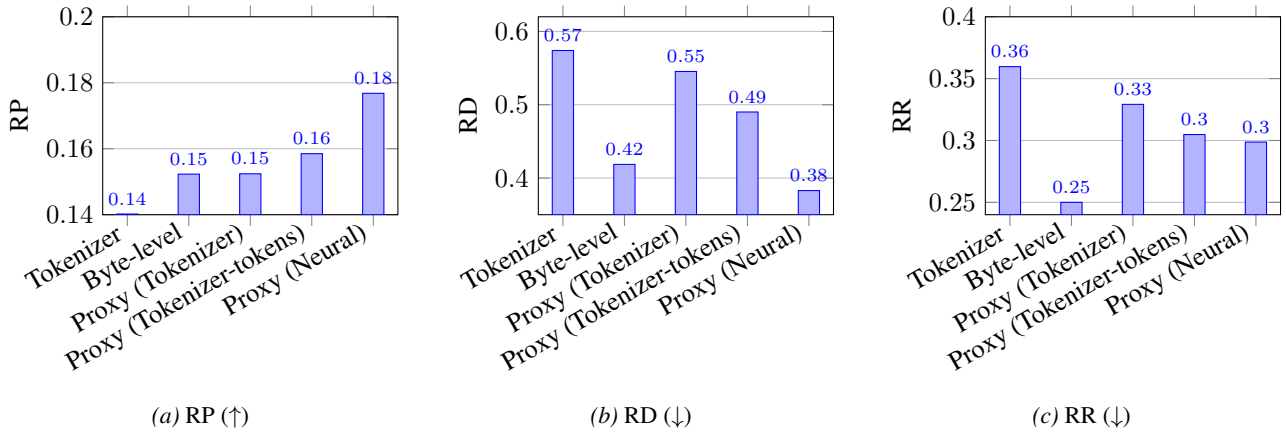


Figure 16. Docstring perturbations.

schedule. Final-checkpoint pass@1 (Table 1) remains the primary signal, with the FLOPs/data plots showing the trajectory during training.

Scaling Behavior. Figures 19a to 19d visualize performance under matched FLOPs (left) and matched data (right) for models at 0.5B, 1.5B, 4B, and 7B parameters. At smaller scales (0.5B), proxy-trained models underperform baselines in both regimes; however, as model size increases, proxy compression becomes progressively more competitive, eventually achieving the best of both regimes at 14B scale (Figure 3). This supports our hypothesis that larger models have greater capacity to learn cross-representation alignment for effective transfer.

Longer Training Horizons. We also examine whether the conventional wisdom, where byte-level models are more data-efficient, tokenizer-based models are more compute-efficient, holds under different training schedules. Figures 20 and 21 plot performance over longer training runs for 1.5B and 7B models, respectively. Notably, byte-level models do not consistently outperform tokenizer-based models under matched data in this regime. This suggests that the data-efficiency advantage of byte-level training might diminish given sufficient training data: the additional compute spent per byte yields diminishing returns, and compression remains beneficial for long-horizon training schedules. However, our proxy compression training still preserves the trend as observed in §3.2.

Related Observations. Our findings align with prior work on the nuanced relationship between compression rate and performance. SuperBPE (Liu et al., 2025) found that higher compression rates do not necessarily improve performance despite consuming more data per step, and discusses the associated trade-off between effective context size and training

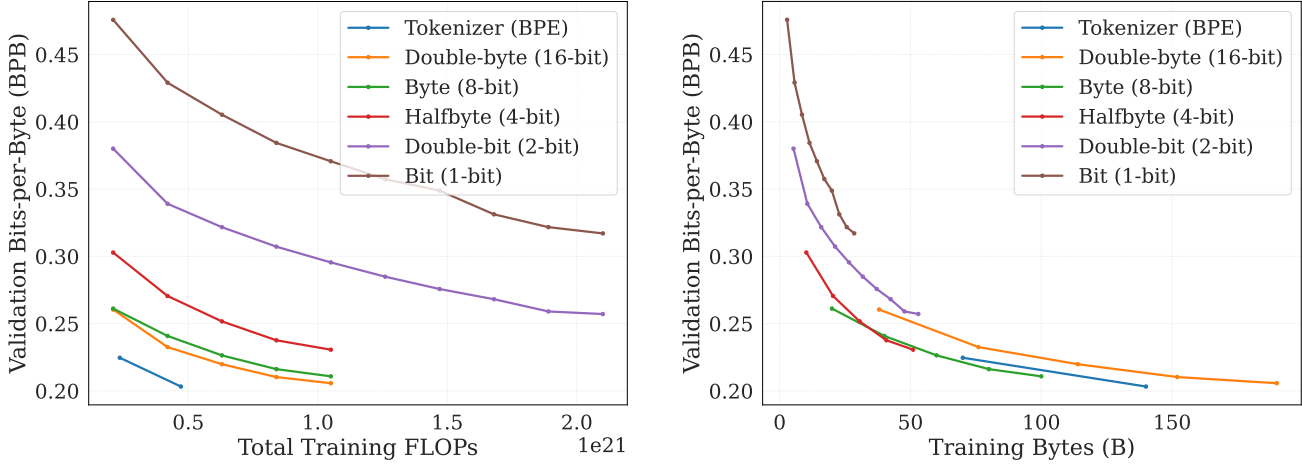


Figure 17. Validation BPB performance of different data representations spanning tokens to bits, under FLOPs-matched (left) and data-matched (right) comparison.

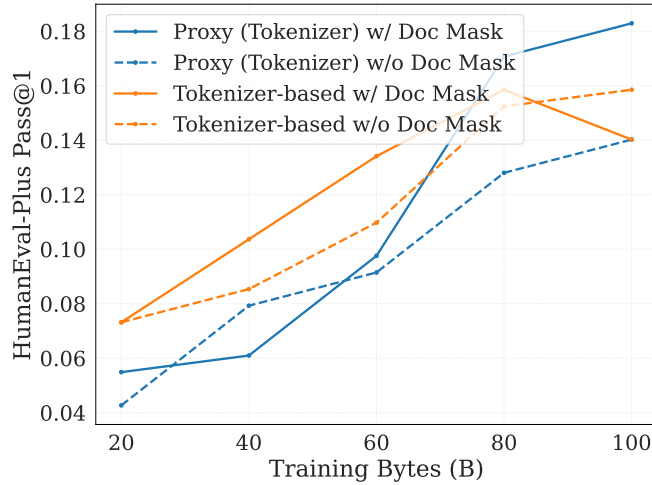


Figure 18. Ablation on document-boundary attention masking.

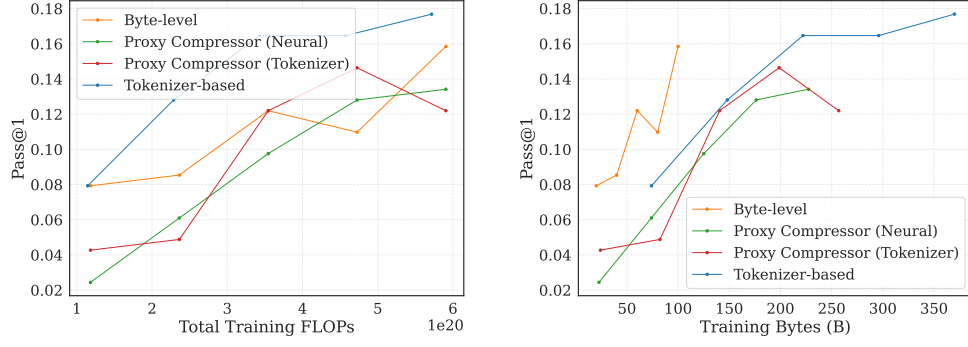
steps.⁴ Possible explanations: (i) higher compression produces harder tokens for the model to learn and predict; or (ii) lower compression yields more optimization steps and thus more compute per byte, allowing better fitting of the data distribution.

D.8. Pairing Strategy Ablation

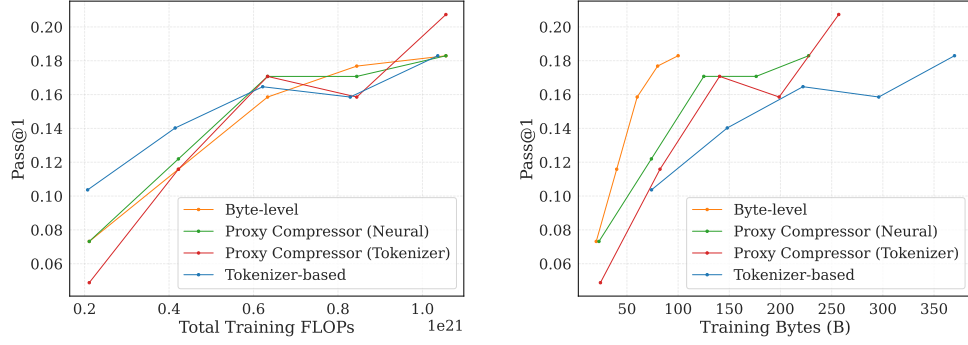
As described in §2.1, we optionally pair compressed and raw views of the same document during training to encourage explicit cross-representation alignment. Table 13 ablates this design choice along three axes: (i) *pairing strategy*: whether pairs are used throughout training (Always-on), only during warmup (Warmup-only), or not at all (None); (ii) *pairing order*: whether raw precedes compressed ($R \rightarrow C$), compressed precedes raw ($C \rightarrow R$), or order is randomized ($R \leftrightarrow C$); and (iii) *rate warmup*: whether the mixing rate r is gradually increased from 0.4 to 0.9 over the first 10K steps.

We observe that *Warmup-only* pairing with randomized order achieves the best downstream performance (20.1% pass@1). This aligns with findings in §3.3: while Always-on pairing yields near-perfect in-context translation, it reduces the effective compression rate by duplicating data as pairs. Warmup-only pairing provides sufficient signal for the model to learn cross-representation alignment early in training, then reverts to independent sampling to maximize data efficiency. Pairing order has a modest effect, with $C \rightarrow R$ slightly under-performing other orderings. Rate warmup provides marginal gains on the performance.

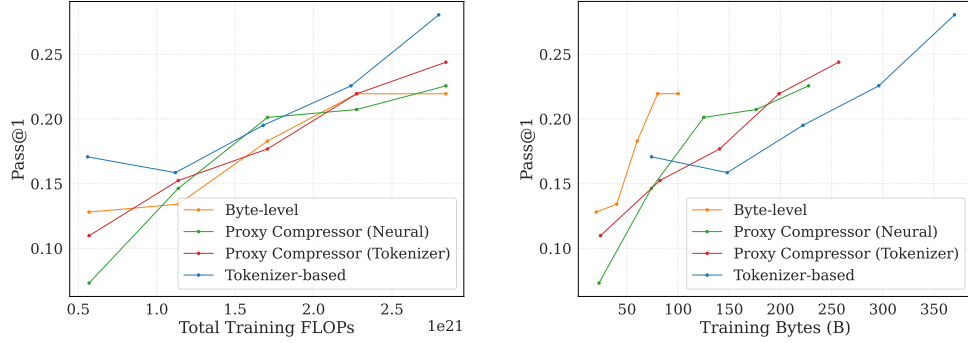
⁴See https://superbpe.github.io/faq.html#context_adjustment for more details.



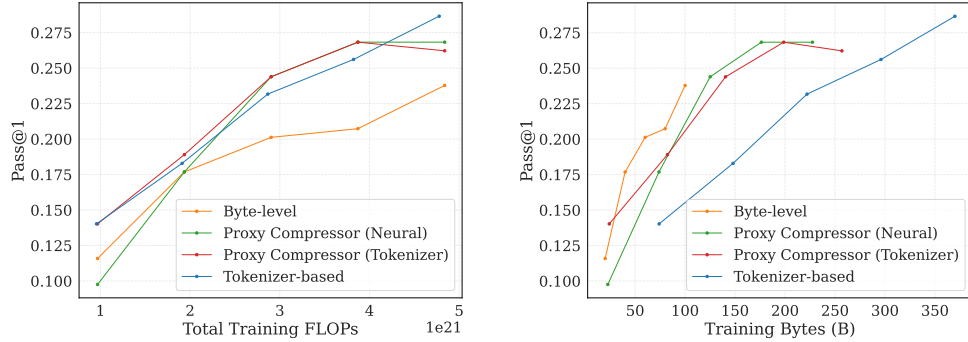
(a) Pass@1 performance on HumanEval-Plus for 0.5B models.



(b) Pass@1 performance on HumanEval-Plus for 1.5B models.



(c) Pass@1 performance on HumanEval-Plus for 4B models.



(d) Pass@1 performance on HumanEval-Plus for 7B models.

Figure 19. Pass@1 performance on HumanEval-Plus under different input representations and model sizes, compared as a function of training FLOPs (left in each row) and amount of training data (right in each row).

Table 13. Ablation of rate warmup and pairing design for proxy compressor training. We report HumanEval-Plus pass@1 and pass@10 on 1.5B models with tokenizer-based proxy compressors. $R \leftrightarrow C$ indicates random ordering with equal probability of $R \rightarrow C$ and $C \rightarrow R$, while $R \rightarrow C$ and $C \rightarrow R$ enforce a fixed order, where $R \rightarrow C$ means that the raw sequence precedes the compressed sequence in the input.

Pairing Strategy	Pairing Order	Rate Warmup	Pass@1	Pass@10
None	—	✗	16.7	25.7
None	—	✓	16.0	27.6
Always-on	$R \leftrightarrow C$	✗	16.6	27.2
Always-on	$R \leftrightarrow C$	✓	16.1	27.5
Always-on	$R \rightarrow C$	✓	16.5	27.0
Always-on	$C \rightarrow R$	✓	16.1	25.6
Warmup-only	$R \leftrightarrow C$	✓	20.1	28.7

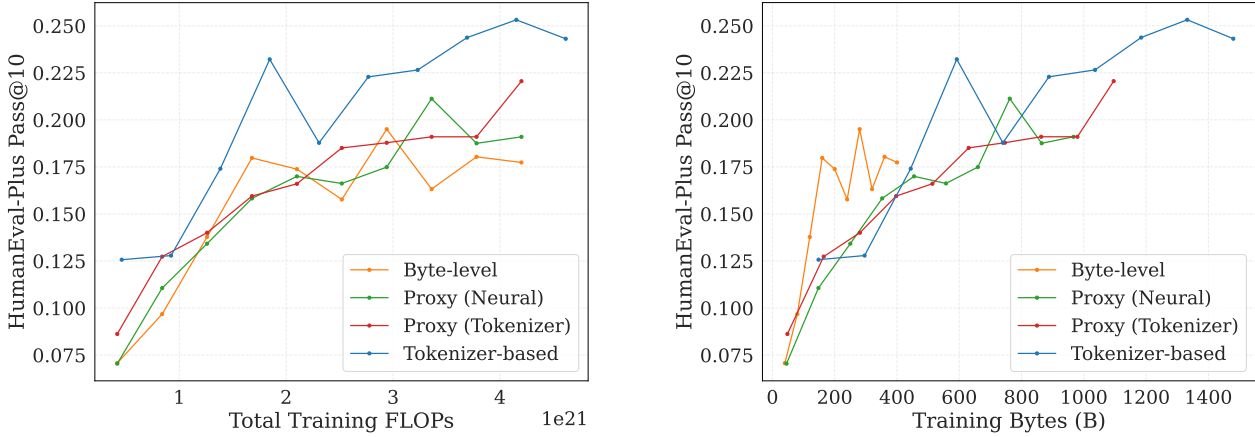


Figure 20. HumanEval-Plus pass@10 for 1.5B models under extended training versus training FLOPs (left) and data consumed (right). We report pass@10 for stability across checkpoints during pretraining.

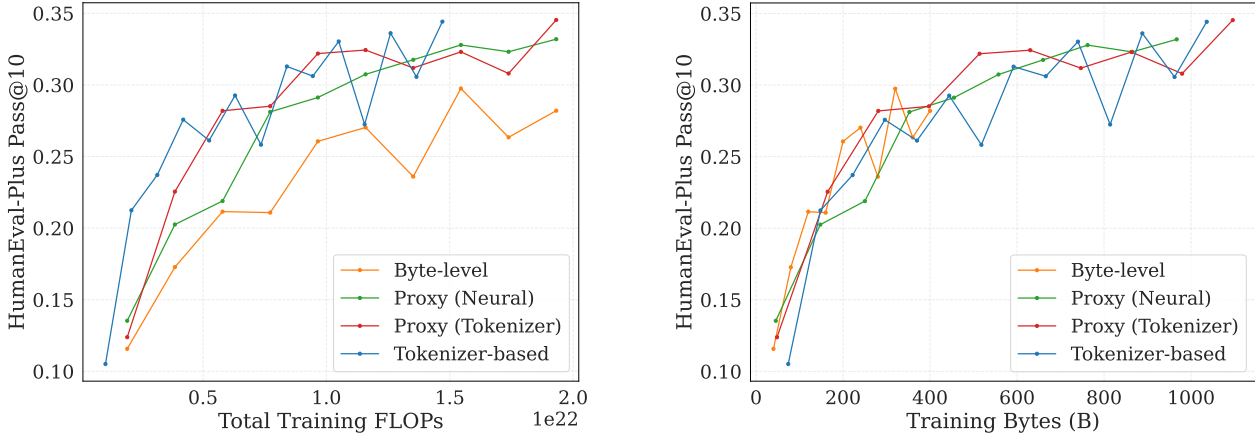


Figure 21. HumanEval-Plus pass@10 for 7B models under extended training versus training FLOPs (left) and data consumed (right). We report pass@10 for stability across checkpoints during pretraining.

D.9. Transfer Strength under Controlled Raw-Byte Exposure

To isolate the effect of transfer from the effect of increased data volume, we compare two training configurations with matched raw-byte exposure but different total training bytes. Specifically, we train 1.5B models on: (i) 95% tokens + 5% raw bytes for 48K steps, and (ii) 90% tokens + 10% raw bytes for 25K steps. Both configurations expose the model to approximately 9.1B raw bytes, but the first sees more total data due to additional compressed bytes.

Table 14. Effect of mixing rate under fixed raw-byte exposure for 1.5B models. Performance is measured with pass@1 rates on HumanEval-Plus.

Mixing Scheme	Training Steps	Raw bytes Seen	Pass@1 (Inference Mode)	
			Bytes	Tokens
95% Tokens + 5% Bytes	48000	9.1B	15.9	15.2
90% Tokens + 10% Bytes	25000	9.1B	12.2	10.4

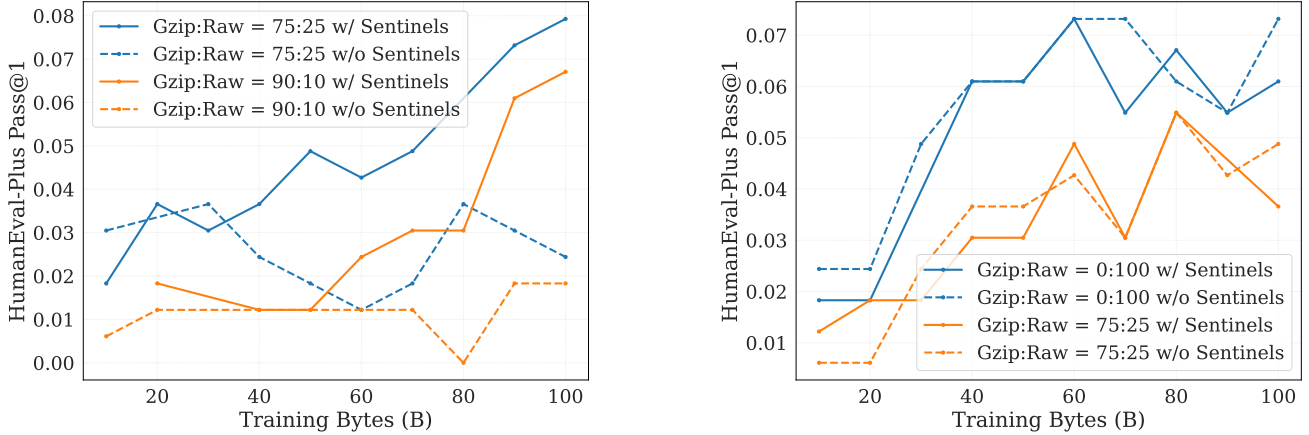


Figure 22. Ablation on format sentinels with gzip proxy compression at 1.5B (left; Python corpus) and 7B (right; full corpus) scales.

Table 14 shows that under matched raw-byte exposure, increasing total training bytes via compressed data yields higher downstream performance on byte-level inference. This confirms *positive transfer* from compressed to raw representations: the model benefits from more compressed training data even when evaluated purely on raw bytes. The same pattern holds for token-level inference (15.2% vs. 10.4%), though the absolute performance is lower than byte-level interface.

D.10. On the Effect of Format Sentinels

Format sentinels (§2.1) explicitly demarcate representation boundaries within packed sequences. We ablate their effect across model scales using preliminary gzip-based proxy compression. As shown in Figure 22, sentinels significantly improve performance at the 1.5B scale, but their benefit becomes marginal at 7B. We hypothesize that larger models can more readily distinguish between representations without explicit markers. Additionally, when training pure byte models (100% raw-format data), we observe no clear performance differences from sentinels, which is expected since no representation boundaries exist. Based on these findings, we enable format sentinels by default, as they provide benefits at smaller scales with negligible overhead at larger scales.

D.11. Prompt Boundary Robustness

A central motivation for byte-level modeling is to avoid tokenizer artifacts such as prompt boundaries, where minor whitespace or formatting changes at the end of the prompt can shift the tokenization decisions and drastically alter model behavior (Microsoft, 2023; Lundberg & Ribeiro, 2023; Athiwaratkun et al., 2024; Hayase et al., 2025). §3.5 evaluates broad robustness via ReCode; here we complement that study with a targeted probe of prompt-boundary sensitivity. We construct 8 perturbations each appending a semantically neutral suffix (whitespace, newlines, or tabs) to a HumanEval prompt (164 problems) after stripping any existing trailing whitespace, plus an unperturbed baseline (P1 in Table 15). None of these suffixes alters the semantics of the prompt, but they can lead to different tokenization boundaries, potentially producing entirely different leading token sequences at the start of generation. We evaluate the same 7B model checkpoints as in Table 1.

The tokenizer-based model exhibits severe sensitivity to prompt boundary perturbations: on HumanEval, its pass@1 rate ranges from 0.00% to 34.15%, with two suffixes, P2 (" \n") and P8 (" \n\n"), collapsing generation to near-zero performance. Its standard deviation across the nine prompt variants is 14.05% on HumanEval and 12.12% on HumanEval-

Table 15. Prompt-boundary robustness results on HumanEval (HE) and HumanEval-Plus (HE+) with 7B models. We report pass@1 (%). Each row appends a semantically neutral suffix to the prompt after stripping trailing whitespace; P1 is the unperturbed baseline.

ID	Description	Suffix	Proxy (Tokenizer)		Byte-level		Tokenizer-based	
			HE	HE+	HE	HE+	HE	HE+
P1	Baseline	(none)	31.71	26.83	27.44	24.39	34.15	29.88
P2	Space + newline	" \n"	32.32	26.22	26.83	23.78	0.61	0.61
P3	Newline	"\n"	34.15	27.44	26.83	23.78	3.05	3.05
P4	Newline + 1 space	"\n "	31.71	25.61	27.44	23.78	27.44	23.78
P5	Newline + 2 spaces	"\n "	32.32	27.44	28.05	24.39	26.83	23.11
P6	Newline + 3 spaces	"\n "	32.32	25.00	27.44	23.78	33.54	29.88
P7	Newline + 4 spaces	"\n "	32.32	26.22	27.44	24.39	25.61	20.12
P8	Double newline	"\n\n"	31.10	23.78	28.05	24.39	0.00	0.00
P9	Newline + tab	"\n\t"	33.54	25.61	25.61	21.34	34.15	29.27
Mean			32.39	26.02	27.24	23.78	20.60	17.74
Std. dev.			0.88	1.11	0.70	0.91	14.05	12.12

Plus. Operating on raw bytes at inference, the proxy-trained model treats appended characters simply as additional byte input and avoids the issue of tokenization boundaries by construction. It maintains a 32.39% mean pass@1 on HumanEval with a standard deviation of only 0.88%, and its average performance even slightly exceeds the unperturbed baseline. The pure byte-level model is similarly robust and stable, as expected, but trails the proxy model in average performance, illustrating that proxy compression effectively combines compressed-training efficiency with byte-level inference robustness. These results directly support the prompt-boundary aspect of the motivation in §1.

D.12. Natural Language Experiments

Setup. To validate the effectiveness of proxy compression beyond code, we train 1.5B models on the Nemotron-CLIMB corpus (Diao et al., 2025) under the same architecture, optimizer, and compute-matched protocol as in §3.1, swapping only the training corpus. Each run uses 50K training steps at a batch size of 0.5M symbols, totaling 25B symbols per run, where symbols denote tokens or bytes depending on the input representation. We compare a tokenizer-based baseline, a raw byte-level baseline, and proxy compression with a tokenizer compressor at $r = 0.9$. We evaluate on 9 standard natural language understanding benchmarks following the OLMES setup (Gu et al., 2024).

Table 16. Downstream task accuracy on natural language understanding benchmarks for 1.5B models under compute-matched training. Proxy (Tokenizer) is evaluated on raw bytes at inference.

Model	ARC-c	ARC-e	BoolQ	CSQA	HellaSwag	OBQA	PIQA	SIQA	WinoGrande	Avg.
Tokenizer-based	37.1	67.3	63.3	47.9	40.4	40.2	68.5	45.3	53.7	51.5
Proxy (Tokenizer)	34.9	65.4	59.5	45.7	42.9	41.2	68.2	43.6	51.5	50.3
Byte-level	35.8	61.4	56.6	38.9	44.0	34.0	67.1	43.1	53.0	48.2

Table 16 reports task accuracy under different approaches. With raw-byte exposure on only 10% of training samples, proxy compression improves over the byte-level baseline by +2.1 average points (50.3 vs. 48.2), closing roughly 64% of the gap between the byte-level and tokenizer-based methods. In addition, it slightly underperforms the tokenizer baseline (51.5), qualitatively consistent with the trend observed in our code experiments at 1.5B, where the gap gradually narrows at larger scales. We also tested whether byte-level capability can be unlocked by late raw-byte finetuning. Starting from a tokenizer-only checkpoint and finetuning for 5k steps on raw bytes improves byte inference from 36.8 to 40.7 average accuracy, but remains far below proxy training at 50.3; after finetuning, token inference also drops to 37.2, suggesting catastrophic forgetting. This supports mixed-representation training during pretraining rather than late conversion. We leave validating the scaling behavior for natural language tasks to future work.